



ADDIS ABABA UNIVERSITY
FACULTY OF NATURAL AND COMPUTATIONAL
SCIENCES

DEPARTMENT OF COMPUTER SCIENCE

**KULLI: P2P Logistics Mobile Application On Demand
Trucking Service**

FINAL PROJECT I BY:

Bereket Berehanu	NSR/6732/14
Chris Tedla	UGR/2624/15
Dawit TekleBirhan	UGR/0426/15
Lamesgnew Awdro	NSR/4291/14

PROJECT ADVISOR: **Dr. Amsale Z.**

June, 2026

iii. DECLARATION

This is to declare that this project work which is done under the supervision of **Dr. Amsale Z.** and having the title **KULLI: P2P Logistics Mobile Application On Demand Trucking Service** is the sole contribution of:

**Bereket Berehanu
Chris Tedla
Dawit Teklebirhan
Lamesgnew Awdro**

No part of the project work has been reproduced illegally (copy and paste) which can be considered as Plagiarism. All referenced parts have been used to argue the idea and have been cited properly. We will be responsible and liable for any consequence if violation of this declaration is proven.

Date: june 28, 2026

Group Members:

**Bereket Berehanu
Chris Tedla
Dawit Teklebirhan
Lamesgnew Awdro**

Full Name

Bereket Berehanu

Signature

Bereket

iv. CERTIFICATE

I certify that this BSc **Final year project** deliverable entitled **KULLI: P2P Logistics Mobile Application On Demand Trucking Service** by:

Bereket Berehanu

Chris Tedla

Dawit Teklebirhan

Lamesgnew Awdro

is approved by me for submission. I certify further that, to the best of my knowledge, the report represents work carried out by the students.

Jan 28, 2026

Date

Dr. Amsale Z.

Name and Signature of Supervisor

ABSTRACT

The Kuli project introduces a peer-to-peer mobile marketplace to resolve the fragmentation and pricing ambiguity currently found in Addis Ababa's informal truck logistics sector. Developed with a React Native frontend and a modular Node.js backend, the system connects clients with verified truck owners using proximity-based matching and transparent cost estimation. It features secure multi-role authentication and a call-assisted booking service to bridge the gap for users with limited digital literacy. Ultimately, the platform fosters trust and economic growth by formalizing service discovery for households and independent logistics providers.

Acknowledgment

I am profoundly grateful to my academic advisor for the invaluable guidance and feedback that directed the development of this project. I also wish to thank the team at Family Movers for providing the operational insights and workflow validation essential to aligning the system with local industry practices. My thanks extend to the independent truck owners and stakeholders who participated in surveys, providing the primary data that grounded our design decisions. Lastly, I appreciate my family and colleagues for their unwavering support during the 32-week roadmap of this project.

TABLE OF CONTENT

iii. DECLARATION	1
iv. CERTIFICATE	3
TABLE OF CONTENT	7
ACRONYMS AND ABBREVIATIONS	9
List of Tables	11
List of Figures	12
1. PROJECT PROPOSAL	14
1.1. Introduction	14
1.2 Statement of the Problem and Justification	14
1.3 Project Objective	14
1.3.1 General Objective of the System	14
1.3.2 Specific Objectives of the System	15
1.4 Scope of the Project	15
1.5 System Development Methodology	16
1.5.1 Investigation (Fact-Finding) Methods	16
1.5.1.1 Stakeholder Identification and Selection	16
1.5.1.2 Surveys and Questionnaires	17
1.5.1.3 Secondary Data and Existing Survey Analysis	17
1.5.2 System Development Tools	17
1.5.2.1 System Architecture	17
1.5.2.2 Development Tools and Technologies	18
1.5.2.3 Testing and Quality Assurance	18
1.6 Significance of the Project	18
1.7 Beneficiaries	18
1.8 Time Schedule	20
2. REQUIREMENT ANALYSIS	21
2.1. Introduction	21
2.2. Current System	21
2.2.1. Major Functions of the Current System	21
2.2.2. Problems of the Existing System	21
2.3. Requirement Gathering	22
2.3.1. Requirement Gathering Methodologies	22
2.3.2. Results Found	22
2.4. Proposed System	23
2.4.1. Overview	23
2.4.2. Functional Requirements	26
2.4.3. Non-Functional Requirements	26
	29

2.5. Constraints / Pseudo Requirements	29
2.6. System Model	31
2.6.1. Scenario	31
2.6.2. Use Case Model	37
2.6.2.1. Use Case Diagram	39
2.3.6.2.2 Description Of Use Case Model	40
2.6.3 Object Model	52
2.6.3.1 Data Dictionary	52
2.6.3.2 Class Modeling	53
2.3.6.4 Dynamic Modeling	55
2.3.6.4.1 Sequence Diagrams	55
2.6.4.2 Activity Diagrams	59
2.6.4.3 State Charts	63
2.6.5 Client Interface	65
3. SYSTEM DESIGN	66
3.1. Introduction	66
3.2. Current Software Architecture	66
3.3. Proposed Software Architecture	67
3.3.1. Overview	67
3.3.2. Subsystem Decomposition	69
3.3.3. Hardware-Software Mapping	70
3.3.4. Persistent Data Management	72
3.3.4.1. Description of Data Schemes	72
3.3.4.2. Selection of Database	72
3.3.4.3. Description of Database Encapsulation	73
3.3.4.3. Object Diagram	73
3.3.5. Access Control and Security	76
3.3.6. Subsystem Services	78
3.4.Detailed Class Diagram	80
3.5. Packages	82
3.5.1 Package Diagram	83
4.5.1 Mapping Models to Code	85
4.5.2 Source Codes of Major Classes, Packages, or Interfaces	92
4. TEST PLAN DOCUMENT	102
4.2 Purpose	103
4.3. Features to be tested/not to be tested	103
4.2.2 Features Not to Be Tested in This Version	105
4.3. Pass/ Fail criteria	106
	107
4.4. Approach / Strategy	108

4.4.1 Overall Testing Approach	108
4.4.2 Tools	109
4.4.3 Metrics Collected	110
4.4.4 Configuration Management	111
4.4.5 Configurations Tested	112
4.4.6 Regression Testing Levels	112
4.4.7 Untestable or Unclear Requirements	113
4.4.8 Coverage Requirements	114
4.4.9 Special Testing Requirements	115
4.5. Test cases with specifications	115

ACRONYMS AND ABBREVIATIONS

ACL: Access Control List

AES: Advanced Encryption Standard

AG: Access & Gateway

API: Application Programming Interface

AWS: Amazon Web Services

BP: Booking & Pricing

BSc: Bachelor of Science

CI/CD: Continuous Integration / Continuous Deployment

CRUD: Create, Read, Update, Delete

ERD: Entity Relationship Diagram

ETA: Estimated Time of Arrival

FD: Feedback & Disputes

FR: Functional Requirement

GIS: Geographic Information System

GPS: Global Positioning System

HTTPS: Hypertext Transfer Protocol Secure

IAM: Identity & Access Management

ID: Identity Provider

JSON: JavaScript Object Notation

JWT: JSON Web Token

KMS: Key Management Service

LO: Live Operations

MFA: Multi-Factor Authentication

NFR: Non-Functional Requirement

ODM: Object-Document Mapper

OTP: One-Time Password

P2P: Peer-to-Peer

PM: Financial Settlement / Payment Manager

QA: Quality Assurance

RBAC: Role-Based Access Control

REST: Representational State Transfer

SMS: Short Message Service

SRS: System Requirements Specification

SU: System Utilities

TCP: Transmission Control Protocol

TLS: Transport Layer Security

TR: Truck Registry

UA: User Accounts

UC: Use Case

UI/UX: User Interface / User Experience

USSD: Unstructured Supplementary Service Data

List of Tables

Table 2.1: Create Account —	26
Table 2.2: Login —	27
Table 2.3: Register Vehicle —	28
Table 2.4: Send KULI Request —	30
Table 2.5: Accept KULI Request	31
Table 2.6: Use Case Model —	32

KULLI: P2P Logistics Mobile Application On Demand Trucking Service

Table 2.7: UC-001 —	35
Table 2.8: UC-002 —	36
Table 2.9: UC-003 —	37
Table 2.10: UC-004 —	38
Table 2.11: UC-005 —	39
Table 2.12: UC-006 —	40
Table 2.13: UC-007 —	41
Table 2.14: UC-008 —	42
Table 2.16: UC-09 —	43
Table 2.17: UC-010 —	44
Table 2.18: UC-011 —	45
Table 2.19: UC-012 —	46
Table 2.20: Data Dictionary —	47
Table 3.1: Access Control List (ACL) Table —	71
Table 3.2: subsystem [Domain Package Mapping] —	77
Table 3.2: subsystem	82
Table 5.1: tested feature	108
Table 5.2: not tested features	109
Table 5.5: pass/fail criteria	110
Table 4.5-1: Test case for Authentication	118
Table 4.5-2: Test case for Role Routing	119
Table 4.5-3: Test case for Client Request	120
Table 4.5-4: Test case for Owner Vehicle	122
Table 4.5-5: Test case for Offer and Trip Execution	123

Table 4.5-6: Test case for Tracking and Messaging	124
Table 4.5-7: Test case for Cancellation	125
Table 4.5-8: Test case for Payment	126
Table 4.5-9: Test case for Rating and Report	127
Table 4.5-10: Test case for Notifications	128
Table 4.5-11: Test case for UI/UX Regression	128

List of Figures

Figure 2.1: Use Case Diagram	34
Figure 2.2: Class Model diagram —	49
Figure 2.3: Authentication Diagram —	50
Figure 2.3: Authentication Diagram —	50
Figure 2.4: Kuli Request —	51
Figure 2.5: Report Truck Owner —	52
Figure 2.6: Vehicle Approval —	53
Figure 2.7: Assisted Booking —	54
Figure 2.8: Registration —	55
Figure 2.9: Vehicle Approval —	56
Figure 2.10: Search Nearby Vehicle —	57
Figure 2.11: Ticket State Chart —	58
Figure 2.12: Kuli State Chart —	59
Figure 2.13: Vehicle State Chart —	59

Figure 2.14: UI Template —	60
Figure 3.1: Architectural design diagram —	63
Figure 3.2: Component Diagram —	65
Figure 3.3: Deployment Diagram —	66
Figure 3.4: Snapshot of a Matched Service Diagram —	68
Figure 3.5: Snapshot of Vehicle Onboarding Diagram —	69
Figure 3.6: Snapshot of Assisted Booking Diagram —	70
Figure 3.7: Accountability, Trust & Dispute Resolution Diagram —	71
Figure 2.1.4.1: User (Base Class) detailed class diagram —	75
Figure 3.8: Client Entity detailed class diagram —	75
Figure 3.9: Truck Owner Entity —	76
Figure 3.10: KULI (Logistics Request) Entity —	76
Figure 3.12: Vehicle Entity —	77
Figure 4.5.3-1: Mobile login page	96
Figure 4.5.3-2: Client quote creation screen	98
Figure 4.5.3-3: Candidate vehicle selection	98
Figure 4.5.3-4: Truck owner dashboard	99
Figure 4.5.3-6: rating/report screen	100
Figure 4.5.3-7: Admin dashboard overview screen	101
Figure 4.5.3-8: Admin vehicle verification screen	102
Figure 4.5.3-9: Admin pricing or reports management screen	103
Figure 4.5.3-10: Hotline/support ticket management screen	104

1. PROJECT PROPOSAL

1.1. Introduction

Logistics services that require truck-based transportation are a frequent necessity for individuals and small businesses. However, many users continue to face significant challenges in securing reliable, affordable, and transparent trucking services for transporting goods and personal belongings. In most cases, clients depend on informal networks or costly service providers, which often result in stress, unpredictable pricing, limited service availability, and avoidable delays. At the same time, independent truck owners struggle to consistently find customers due to limited visibility, weak marketing channels, and the absence of structured digital platforms that connect them with potential clients.

To address these challenges, this project proposes a peer-to-peer (P2P) mobile application, named **Kuli**, that connects clients requiring truck-based logistics services including household relocation as a primary use case with verified independent truck owners, creating a simple, secure, and transparent digital marketplace that optimizes booking, communication, and service delivery.

1.2 Statement of the Problem and Justification

Clients frequently experience difficulty finding trustworthy and available truck owners, comparing prices and service quality, scheduling transportation services efficiently, and communicating clearly to ensure reliable delivery. Similarly, independent truck owners face challenges such as limited visibility to potential customers, inconsistent job opportunities, and the lack of a centralized platform to manage bookings and service requests. These issues result in unreliable service delivery, unclear pricing, frequent delays, and frustration for both parties.

Therefore, a centralized mobile application is needed to improve trust, efficiency, and accessibility in truck-based logistics services, particularly for household relocation and similar transportation needs.

1.3 Project Objective

1.3.1 General Objective of the System

The general objective of this study is to design a peer-to-peer mobile application that facilitates efficient, reliable, and transparent truck-based logistics services by connecting clients with verified independent truck owners through a centralized digital platform.

1.3.2 Specific Objectives of the System

- To analyze existing truck-based logistics and household relocation processes and identify key challenges related to trust, accessibility, pricing transparency, and communication.
- To design a secure registration and authentication mechanism that verifies both clients and independent truck owners before they access the platform.
- To enable clients to create detailed transportation requests, including pickup location, destination, distance, type and volume of items, preferred service date, and required truck type.
- To design a proximity-based listing mechanism that allows clients to view available truck owners based on geographic closeness to the pickup location, reducing delays and improving service efficiency.
- To allow independent truck owners to browse and accept suitable transportation requests based on availability, vehicle capacity, and location.
- To design a service ranking and filtering system that lists truck owners based on non-price criteria such as user ratings, vehicle type, proximity, and past service performance.
- To incorporate a rating and review system that enables clients to evaluate completed services and promotes accountability and trust within the platform.
- To provide notifications that keep users informed of job updates and service status changes.
- To design an administrative dashboard for user management, operational oversight, and platform quality control.

1.4 Scope of the Project

The proposed platform will initially operate within a single city, specifically Addis Ababa, and will target two primary user groups: clients seeking truck-based logistics services and independent truck owners providing transportation services. Household relocation will serve as the primary use case of the system, while the platform remains flexible enough to support other transportation needs such as small business deliveries and equipment transport.

Core system features will include secure user registration and authentication, service request posting and acceptance, internal messaging for coordination, a rating and review mechanism to enhance trust, basic price estimation, and an administrative dashboard for platform monitoring. Additionally, the system will support a call-assisted booking service for users without smartphones or limited digital literacy, allowing call-center assistants to create and manage requests on their behalf.

The initial version of the platform will have certain limitations. Real-time GPS tracking of trucks will not be implemented in the first phase, and integrated digital payment services may be introduced in later versions. The platform will function solely as a connector between clients and truck owners and will not be responsible for disputes, damages, or physical handling of goods. Advanced pricing optimization and machine-learning-based automation will also be excluded from the first phase.

1.5 System Development Methodology

The study combines primary data collected from system stakeholders with secondary data sourced from existing logistics platforms and academic literature. Basic analytical techniques, including descriptive analysis of survey data and system modeling, will be used to transform collected data into system requirements and design decisions, ensuring that the system addresses real user needs.

1.5.1 Investigation (Fact-Finding) Methods

Information will be gathered from multiple sources to understand user needs and logistics workflows. This includes individual clients, independent truck owners, and workers or truck owners employed at **Family Movers**, a well-known local moving company in Addis Ababa, to gain practical insights into real-world transportation operations and challenges.

1.5.1.1 Stakeholder Identification and Selection

The primary stakeholders for the system are:

- Clients (renters, students, and small businesses)
- Independent Truck Owners
- Call-Center Assistants
- System Administrators

A purposive sampling strategy will target these groups efficiently. Clients will be engaged through digital channels, such as local Telegram housing groups and university networks, to gather requirements from frequent movers. **Family Movers** will serve as a supporting subject matter expert, providing operational insights, pricing information, and workflow validation to align the system with industry practices. Independent Truck Owners will be interviewed at high-density locations, e.g., near furniture shops or electronics stores, to understand requirements for transporting large items.

1.5.1.2 Surveys and Questionnaires

Structured surveys will be conducted both online and offline to gather data on service demand, current logistics practices, challenges faced, preferred booking methods.

1.5.1.3 Secondary Data and Existing Survey Analysis

Secondary data will be collected from academic journals, research reports, and studies of existing logistics platforms such as Uber, Bolt, and other open-source or publicly documented systems (e.g., OpenStreetMap, OpenTripPlanner, Open Logistics datasets) to support and validate system design decisions.

1.5.2 System Development Tools

1.5.2.1 System Architecture

The system will adopt a client–server architecture with a modular monolithic backend, separating core functions such as authentication, job management, matching, messaging, notifications, and administration. This architecture is selected to simplify development and deployment while ensuring clear separation of concerns, making the system easier to understand, maintain, and extend during the project lifecycle.

This architectural style is chosen because it provides a balance between simplicity and scalability. It reduces development and operational complexity compared to microservices while maintaining clear separation of concerns within the system. The modular structure also allows future migration to microservices if system scale and usage increase.

1.5.2.2 Development Tools and Technologies

- Frontend: React Native for cross-platform mobile development
- Backend: Node.js with a structured framework to support modularization
- Database: MongoDB + Geospatial Indexing
- Authentication: Supabase Auth
- UI/UX Design: Figma for interface prototyping
- APIs: RESTful APIs for communication between client and server

1.5.2.3 Testing and Quality Assurance

Testing will include unit, integration, and functional testing using tools such as Jest, Postman, and React Native Testing Library, supported by Git-based version control and code quality tools.

1.6 Significance of the Project

The system improves access to reliable truck-based logistics services by offering verified service providers, transparent pricing, and efficient communication. Features such as proximity-based matching, messaging, ratings, and call-assisted booking enhance trust, inclusivity, and operational efficiency.

The platform also empowers independent truck owners by increasing visibility and enabling consistent job opportunities, contributing to the formalization and growth of small-scale logistics services.

1.7 Beneficiaries

Primary beneficiaries include individuals, households, and small businesses requiring transportation services, particularly for household relocation. The platform provides access to verified truck owners, transparent service options, proximity-based matching, and efficient

communication, which reduces delays, lowers search costs, and minimizes logistics-related stress.

Independent truck owners also benefit from increased visibility, structured and consistent job opportunities, and simplified booking management. The rating and review system helps them build credibility, while proximity-based matching reduces idle time and operational inefficiencies, contributing to more stable income generation.

The product owner (platform operator) benefits through a sustainable revenue model. The platform can generate income by charging a small commission on each successfully completed transaction, service or booking fees, optional premium listings for truck owners, and potential cancellation fees. These revenue streams support system maintenance, operational costs, scalability, and future feature enhancements.

System administrators and support staff benefit from improved tools for user management, service monitoring, and operational oversight, ensuring platform reliability and quality control. Overall, the platform creates economic value for all stakeholders by improving trust, efficiency, and transparency within the local logistics ecosystem.

1.8 Time Schedule

Simplified 32-Week Roadmap

Simplified 32-Week Roadmap chart		weeks(1-32)																															
PROJECT PHASES		weeks(1-8)								weeks(9-16)								weeks(17-24)								weeks(25-32)							
Phase 1: Research and Planning		Define scope, review literature																															
1.1	Proposal & Topic Approval																																
	Week 1 - 2																																
1.2	Comprehensive Literature Review																																
	Week 3 - 6																																
1.3	Requirements Elicitation & Data Gathering																																
	Week 6 - 8																																
Phase 2: Core System Design		Architectural planning and proto																															
2.1	System Requirements Specification (SRS)																																
	Week 9 - 12																																
2.2	Database Design (ERD & Schema)																																
	Week 13 - 15																																
2.3	UI/UX Mockups and Wireframing																																
	Week 15 - 16																																
Phase 3: Development & Backend Logic		Building the core func																															
3.1	API Setup & Authentication Module																																
	Week 17 - 19																																
3.2	Fleet Management Module (CRUD)																																
	Week 19 - 22																																
3.3	Booking/Rental Logic Implementation																																
	Week 22 - 25																																
Phase 4: Frontend & Finalization		Client-side interface, testing,																															
4.1	Customer Web/Mobile Frontend																																
	Week 25 - 28																																
4.2	Admin Dashboard Interface																																
	Week 27 - 29																																
4.3	System Testing & QA																																
	Week 29 - 30																																
4.4	Final Report & Defense Preparation																																
	Week 31 - 32																																

2. REQUIREMENT ANALYSIS

2.1. Introduction

The purpose of the system is to develop a peer-to-peer (P2P) mobile application, Kuli, that facilitates on-demand truck-based logistics services within Addis Ababa. The system serves as a centralized digital marketplace connecting clients who require transportation of goods such as household items, furniture, appliances, and small-scale business deliveries with verified independent truck owners. By enabling proximity-based service discovery, direct communication, transparent service information, and a call-assisted booking option, the system aims to address inefficiencies, trust issues, and pricing ambiguity present in the current logistics market.

2.2. Current System

2.2.1. Major Functions of the Current System

Currently, truck-based logistics services operate through informal and fragmented processes. Clients typically locate truck owners by physically visiting known truck parking areas, relying on word-of-mouth referrals, informal brokers, or personal contacts. Negotiations regarding price, availability, and service details are conducted verbally and often vary significantly depending on negotiation skills, urgency, and perceived demand. Some clients choose established moving companies for reliability; however, these services are generally more expensive and less flexible. These approaches are used not only for household relocation but also for transporting furniture, appliances, and goods for small businesses.

2.2.2. Problems of the Existing System

The existing system presents several challenges:

- **Pricing Inconsistency:** Costs are determined through informal negotiation, leading to unpredictable and sometimes inflated pricing.
- **Difficulty in Service Discovery:** Clients expend considerable time and effort locating available trucks without assurance of success.
- **Lack of Trust and Accountability:** The absence of formal verification exposes clients to risks such as service cancellation, damage, or theft.

- **Operational Inefficiency:** Truck owners experience idle time due to limited visibility, while clients struggle to compare vehicle types and capacities.

2.3. Requirement Gathering

2.3.1. Requirement Gathering Methodologies

A mixed-method approach was used to collect system requirements:

- **Online Surveys:** An online survey was conducted using a purposive sampling strategy to gather requirements from users with prior experience in household relocation and truck-based logistics services. The survey was primarily distributed through local Telegram housing channels and university-related groups, which are commonly used by frequent movers. This approach enabled efficient access to relevant participants and ensured that the collected responses reflected real user challenges and expectations.
- **Informal Interviews and Stakeholder Consultation:** Discussions with independent truck owners and workers or truck owners associated with **Family Movers** (well-known local moving company in Addis Ababa) to understand real operational workflows and constraints.
- **Secondary Data Analysis:** Review of academic studies and publicly available documentation of logistics and ride-hailing platforms (e.g., Uber, Bolt) to benchmark features and best practices.

2.3.2. Results Found

The requirement analysis revealed the following key findings:

- Strong demand for **verified and trustworthy truck owners** to ensure safety and reliability.
- High importance is placed on **accurate and transparent price estimation**, ensuring users receive clear cost estimates based on distance, item type, and truck capacity prior to booking.
- The need for **alternative booking methods** due to varying levels of digital literacy, justifying call-assisted services.

- Preference for **proximity-based matching** to reduce waiting times and transportation delays.

2.4. Proposed System

2.4.1. Overview

The proposed system, **Kuli**, is a mobile-based P2P logistics platform designed to support general truck-based transportation services. It connects clients with verified independent truck owners for multiple logistics use cases, including household relocation, item transportation, and small-scale commercial deliveries. The system supports service request creation, proximity-based matching, communication, notifications, ratings, administrative oversight, and a call-assisted booking mechanism to enhance accessibility.

2.4.2. Functional Requirements

FR-1 User Accounts and Authentication

- **FR-1.1:** The system shall allow users to **register** as one of four roles: **Client**, **Truck Owner**, **Call-Center Assistant**, or **Administrator**.
- **FR-1.2:** The system shall support **email/phone-based registration** and **password** or **OTP** authentication for all roles.
- **FR-1.3:** The system shall support **role-based login** and present role-specific dashboards immediately after authentication.
- **FR-1.4:** The system shall allow users to **reset passwords** and recover accounts via secure OTP or email links.

FR-2 Truck Owner Verification and Onboarding

- **FR-2.1:** The system shall provide a **verification workflow** for truck owners that collects vehicle details, identity documents, and proof of ownership/insurance.
- **FR-2.2:** The system shall allow administrators to **review, approve, or reject** truck owner applications and record the decision with timestamps.
- **FR-2.3:** The system shall prevent unverified truck owners from accepting or appearing in public listings.

- **FR-2.4:** The system shall notify truck owners of verification status changes via in-app notification and SMS/email.

FR-3 Service Request Creation and Management

- **FR-3.1:** The system shall allow clients to **create a logistics service request** specifying: pickup location, destination, item type, item dimensions/volume, weight, preferred pickup date/time, and any special handling instructions.
- **FR-3.2:** The system shall validate required fields and provide inline guidance for acceptable values (e.g., max volume per truck type).
- **FR-3.3:** The system shall allow clients to **cancel** a submitted request within configurable policy windows.

FR-4 Matching, Discovery, and Filtering

- **FR-4.1:** The system shall present a **proximity-based list** of available verified truck owners sorted by distance, availability and aggregate rating..
- **FR-4.2:** The system shall provide filters for **truck type, capacity**.
- **FR-4.3:** The system shall support **automatic matching** that suggests the top N truck owners for a request based on distance, capacity, availability, ratings, and dispute history.

FR-5 Pricing, Quotation, and Payments

- **FR-5.1:** The system shall **calculate and display trip cost estimates** using configurable and market-adjustable pricing rules that consider truck type, load size or volume, travel distance, estimated trip duration, toll charges, prevailing fuel prices, and optional services (e.g., loading assistance), with support for future extension to additional vehicle types.
- **FR-5.2:** The system shall support **multiple payment flows**: pay-on-acceptance, pay-on-delivery, and pay-in-advance (digital payments may be implemented later).
- **FR-5.3:** The system shall record **payment transactions**.

FR-6 Trip Execution and Real-Time Tracking

- **FR-6.1:** The system shall provide **real-time status updates** for each shipment: Requested, Matched, Accepted, En Route to Pickup, Loading, In Transit, Unloading, Completed, Cancelled..
- **FR-6.2:** The system shall allow authorized truck owners or drivers to manually **update predefined shipment statuses** (e.g., Accepted, En Route to Pickup, Loading, Unloading)
- **FR-6.3:** The system shall maintain a **complete event log** for each trip with timestamps.

FR-7 Communication and Notifications

- **FR-7.1:** The system shall provide **in-app messaging** between clients and truck owners with message history tied to the service request.
- **FR-7.2:** The system shall support **call-assisted booking** where authorized call-center assistants can create or modify requests on behalf of clients and record the operator ID.
- **FR-7.3:** The system shall send **notifications** (in-app, SMS, email) for key events: request submission, acceptance, driver en route, arrival, delivery completion, cancellations, and payment receipts.
- **FR-7.4:** The system shall allow users to configure **notification preferences** and opt out of non-essential messages.

FR-8 Ratings, Reviews, and Dispute Resolution

- **FR-8.1:** The system shall allow clients to **submit numeric ratings** and textual reviews for completed trips.
- **FR-8.2:** The system shall **compute an aggregate rating** score for each truck owner based on completed service ratings.
- **FR-8.3:** The system shall make aggregate ratings available to the matching and filtering module to **influence provider ranking and selection**.

- **FR-8.4:** The system shall provide a dispute workflow where clients or truck owners can submit complaints with supporting evidence; administrators can review, mediate, and record outcomes.
- **FR-8.5:** The system shall flag or **penalize truck owners** with unresolved or frequent disputes, affecting their visibility in matching results.

FR-9 Administration and Reporting

- **FR-9.1:** The system shall provide an **administrative dashboard** for user management, service oversight, verification queue, pricing rules, and system health metrics.

FR-10 Truck Owner Dashboard

- **FR-7.1:** The system shall **provide a dashboard** for registered truck owners to manage their activities.
- **FR-7.2:** The system shall allow truck owners to **view and respond to trip requests**, and monitor active and completed trips.
- **FR-7.3:** The system shall allow truck owners to **update trip status** (e.g., Accepted, Loading, In Transit, Completed).
- **FR-7.4:** The system shall allow truck owners to **manage their availability**.
- **FR-7.5:** The system shall allow truck owners to **view their aggregated ratings** .

2.4.3. Non-Functional Requirements

2.4.3.1. Business Rules

- **NFR-1.1 Single Active Request Rule:** A client may have up to **one active pickup request per shipment**; business rules for concurrent requests will be configurable by admin.
- **NFR-1.2 Verification Requirement:** Only **verified truck owners** shall be eligible to receive or accept requests.
- **NFR-1.3 Cancellation Policy:** Cancellation windows, fees, and penalties shall be configurable and enforced by the system.

2.4.3.2. User Interface and Human Factors

- **NFR-2.1 Intuitive Role-Specific UI:** The UI shall be **role-specific**, minimizing visible options to only those relevant to the user's role.
- **NFR-2.2 Mobile-First Design:** The system shall be **mobile-friendly** and usable on standard smartphones with screens down to 4.7 inches.
- **NFR-2.3 Accessibility:** The UI shall meet basic accessibility guidelines (contrast, scalable text, touch target sizes) to support users with limited digital literacy.
- **NFR-2.4 Local Language Support:** The system shall support the primary local language(s) (planned for future implementation) and English for menus and key workflows.

2.4.3.3. Documentation

- **NFR-3.1 User Guides:** Provide **concise user manuals** for Clients, Truck Owners, and Call-Center Assistants with step-by-step workflows and screenshots.
- **NFR-3.2 Technical Documentation:** Provide API documentation, deployment guides, and maintenance procedures for developers and system maintainers.
- **NFR-3.3 Change Log:** Maintain a versioned change log for releases and configuration changes.

2.4.3.4. Hardware Consideration

- **NFR-4.1 Supported Devices:** The mobile application shall support **Android 8+** and **iOS 12+** and degrade gracefully on older devices.
- **NFR-4.2 GPS and Sensors:** The system shall use device GPS for location services and allow manual location entry when GPS is unavailable.
- **NFR-4.3 No Specialized Hardware:** No specialized external hardware shall be required for normal operation.

2.4.3.5. Performance Characteristics

- **NFR-5.1 Response Time:** The system shall respond to user interactions within **3 seconds** under normal load and within **5 seconds** under peak city-wide load.

- **NFR-5.2 Concurrency:** The system shall support concurrent users at a city scale with horizontal scaling options for peak demand.
- **NFR-5.3 Throughput:** The system shall process booking submissions and status updates with a throughput sufficient to handle peak hourly request volumes (to be defined during capacity planning).

2.4.3.6. Error Handling and Extreme Conditions

- **NFR-6.1 Graceful Degradation:** Core features (request creation, status updates, notifications) shall remain available under partial system failures; non-critical features may be temporarily disabled.
- **NFR-6.2 Clear Error Messaging:** The system shall present **user-friendly error messages** with actionable guidance for common issues (invalid input, network loss, authentication failure).
- **NFR-6.3 Retry and Offline Support:** The mobile client shall queue critical actions for retry when connectivity is restored and inform users of offline status.

2.4.3.7. Quality Issues

- **NFR-7.1 Availability Target:** The system shall aim for **99.9% uptime** excluding scheduled maintenance windows.
- **NFR-7.2 Monitoring and Alerts:** Implement monitoring for performance, errors, and security events with automated alerts to administrators.

2.4.3.8. System Modifications

- **NFR-8.1 Modular Architecture:** The system shall be modular to allow incremental feature additions (digital payments, advanced GPS tracking) with minimal downtime.
- **NFR-8.2 CI/CD and Rollback:** Support continuous integration and deployment pipelines with automated tests and safe rollback mechanisms.

2.4.3.9. Physical Environment

- **NFR-9.1 Urban Deployment:** The system shall be optimized for urban environments (Addis Ababa initial deployment) and tolerate variable mobile network quality.
- **NFR-9.2 Data Center and Cloud:** The system shall be deployable to cloud infrastructure with region-aware redundancy.

2.4.3.10. Security Issues

- **NFR-10.1 Authentication and Authorization:** Enforce secure authentication, multi-factor options, and strict role-based access control.
- **NFR-10.2 Data Protection:** Encrypt sensitive data in transit (TLS) and at rest; store minimal personally identifiable information and support data deletion requests.
- **NFR-10.3 Audit Trails:** Maintain tamper-evident audit logs for security-relevant events.

2.4.3.11. Resource Issues

- **NFR-11 .1 Efficient Resource Use:** The system shall minimize bandwidth and storage usage on mobile devices and support incremental data sync.

2.5. Constraints / Pseudo Requirements

The following key real-world constraints represent the major bottlenecks that may affect the system. Each includes an impact description and suggested mitigation:

1. Limited Truck Availability and Uneven Supply

Impact: Peak demand periods may produce long waits and unmet requests, reducing user satisfaction.

Mitigation: Implement surge indicators, waitlists, driver incentives, and partnerships with fleet operators to smooth supply.

2. Verification and Onboarding Delays

Impact: Manual vetting slows driver onboarding and limits available supply during launch.

Mitigation: Combine automated document checks with prioritized manual review and provisional statuses for early visibility.

3. Limited insurance coverage increases financial and legal risk

Impact: Limited insurance coverage increases financial and legal risk.

Mitigation: Require truck owners to provide vehicle insurance where available, disclaim coverage for transported goods, and allow optional cargo insurance if it becomes available in the future.

4. Cash-First Economy and Low Digital Payment Adoption

Impact: Cash settlements complicate commission collection and dispute resolution.

Mitigation: Support cash workflows while incentivizing digital payments and implement reconciliation processes.

5. Network Connectivity and Device Limitations

Impact: Intermittent data and older devices reduce app reliability for a portion of users.

Mitigation: Provide lightweight UI, offline-capable forms for call agents, and SMS/USSD fallbacks for critical notifications.

6. Geolocation and Address Accuracy

Impact: Inaccurate or nonstandard addresses cause misplaced pickups and failed jobs.

Mitigation: Allow manual pin adjustments, require address confirmation, and use call-assisted verification for ambiguous locations.

7. Data Quality and Fraud Risk

Impact: Fake profiles, manipulated ratings, and inaccurate listings undermine trust and platform credibility.

Mitigation: Enforce multi-factor verification, periodic audits, and anomaly detection to flag suspicious activity.

8. Call-Assisted Capacity Limits

Impact: Limited call-center throughput constrains service access for non-digital users.

Mitigation: Scale call team to expected demand, implement templated request flows, and provide supervisors with workload dashboards.

9. Traffic Congestion and Road Conditions

Impact: Urban congestion and poor roads produce unpredictable trip times and higher costs.

Mitigation: Use conservative ETAs, allow driver route adjustments, and provide congestion warnings to clients.

10. Fuel Price Volatility and Operating Cost Sensitivity

Impact: Rapid fuel cost changes can render fixed estimates unprofitable, increasing

cancellations.

Mitigation: Include adjustable cost components, periodic fare reviews, and optional fuel surcharges during price spikes.

2.6. System Model

2.6.1. Scenario

This section contains the major real life scenarios of the system use cases described in a structured flow of events.

P.S In the current section, ‘/’ is used to convey “one of these” while ‘,’ is used to convey “all of them”.

2.6.1-1 Create Account

Table 2.1: Create Account

Scenario ID	SC-001
Scenario Name	Create Account
Participating Actors	Client / Truck Owner
Entry Conditions	Actor has a phone number or an email account
Flow Of Events	1. Actor accesses the application and is directed to the landing page 2. Actor selects the Create Account button to enter the registration page 3. Actor picks the appropriate role from Client / Truck Driver 4. Actor will pick between a phone number, an email

	account, or both with one as primary 5. Actor will enter the other required details, including the password 6. System will verify validity and availability of the entered credentials 7. If successful the system will create the account and login the actor 8. The system redirects to the home page
End Conditions	1. The Account is created and the actor is logged in and redirected to the home page 2. The Credentials are invalid and the actor is prompted to try again
Quality Requirements	1. The actor will input valid credentials

2.6.1-2 Login

Table 2.2: Login

Scenario ID	SC-002
Scenario Name	Login
Participating Actors	Client / Truck Owner / Call Center Assistant
Entry Conditions	Client has been registered and has an account

Flow Of Events	1. Actor accesses the application and selects the Log In button 2. Actor enters the credentials, including the email/phone number and password 3. System validates the account credentials and determines the account type 4. If successful, actor is redirected to the appropriate home page
End Conditions	1. Actor has logged into the system 2. Actor has inputted invalid credentials and is prompted to try again
Quality Requirements	1. Actor remembers and inputs the correct credentials

2.6.1-3 Register Vehicle

Table 2.3: Register Vehicle

Scenario ID	SC-003
Scenario Name	Register Vehicle
Participating Actors	Truck Owner, Admin

Entry Conditions	Actor has an account and has the necessary vehicle documents
Flow Of Events	1. Actor Logs In to the system and clicks the Register Vehicle button 2. Actor is redirected to the vehicle registration page 3. Actor will be choose the vehicle class, upload personal identification, drivers license, vehicle information document, vehicle registration certificate, vehicle insurance, etc and submit 4. System will check for unintelligible or corrupted files 5. If successful, system will send the data for admin review 6. Admin will review the documents and allow vehicle registrations if all requirements are met 7. If successful, Actor has successfully added a vehicle to his account and can operate with it
End Conditions	1. Vehicle is registered to the account 2. Vehicle registry is denied and the reason is provided
Quality Requirements	1. Actor provides legally and digitally valid information and documents

2.6.1-4 Send KULI Request

Table 2.4: Send KULI Request

Scenario ID	SC-004
Scenario Name	Send KULI Request
Participating Actors	Client / Call Center Assistant
Entry Conditions	The actor is logged into the system
Flow Of Events	1. Actor clicks the KULI Request button and is redirected to the sending page 2. Actor inputs the vehicle class of choice, destination, estimated luggage weight and volume, incentive tip amount, and other required inputs and submit 3. System validates input and calculates KULI price 4. System searches and displays the matched and recommended vehicle list sorted by proximity and driver rating 5. Actor selects preferred drivers and sends request
End Conditions	1. Actor successfully sends KULI request 2. Actor does not follow through with the request 3. System does not find available vehicles on standby

Quality Requirements	The actor provides honest and reasonable information
-----------------------------	--

2.6.1-5 Accept KULI Request

Table 2.5: Accept KULI Request

Scenario ID	SC-005
Scenario Name	Accept KULI Request
Participating Actors	Truck Owner
Entry Conditions	1. Actor is Logged In 2. Actor has an active registered vehicle 3. Actor is within a reasonable proximity to the request
Flow Of Events	1. System sends request to the fitting Truck Owners on standby 2. Actor reviews KULI request and contacts Client via contact info 3. If in agreement, Truck Owner accepts the KULI request
End Conditions	1. Actor contacts Client, is in agreement and accepts request 2. Actor contacts Client, is not in agreement and

	rejects request
Quality Requirements	Actor provides a reasonable argument

2.6.2. Use Case Model

Table 2.6: Use Case Model

Use Case ID	Primary Actor	Use Case Name	Functionality
UC-001	Admin, Client, Truck Owner, Call Assistant	Login/Register	Registration and Authentication of respective users
UC-002	Admin	Manage Users/Reports	View reports and Edit/Suspend/Ban/Remove/verify an account
UC-003	Truck Owner	Register/Manage Vehicle	Register Vehicle. Set currently active vehicle and update vehicle information.
UC-004	Client, Call Assistant	Create KULI Request	Create a KULI request based on preferences
UC-005	Truck	Handle KULI	Review, accept/ignore, cancel a KULI request

KULLI: P2P Logistics Mobile Application On Demand Trucking Service

	Owner	Request	
UC-006	Truck Owner	Update KULI Status	Update an accepted KULI status
UC-007	Client	Rate Terminated KULI	Rate Truck Owner on a cancelled or completed KULI
UC-008	Client, Call Assistant	Report Truck Owner	Report truck owner misconduct
UC-009	Admin	Handle Vehicles	Add/Edit/Remove vehicle class and approve vehicle registry
UC-010	Call Assistant	Handle Assisted Booking	Allows customer service to create KULI requests on behalf of clients
UC-011	Call Assistant	Handle Hotline Ticker	Update ticket status from initiation to finalization
UC-012	TruckOwner	Confirm Payment	Confirm calculated payment completion

2.6.2.1. Use Case Diagram

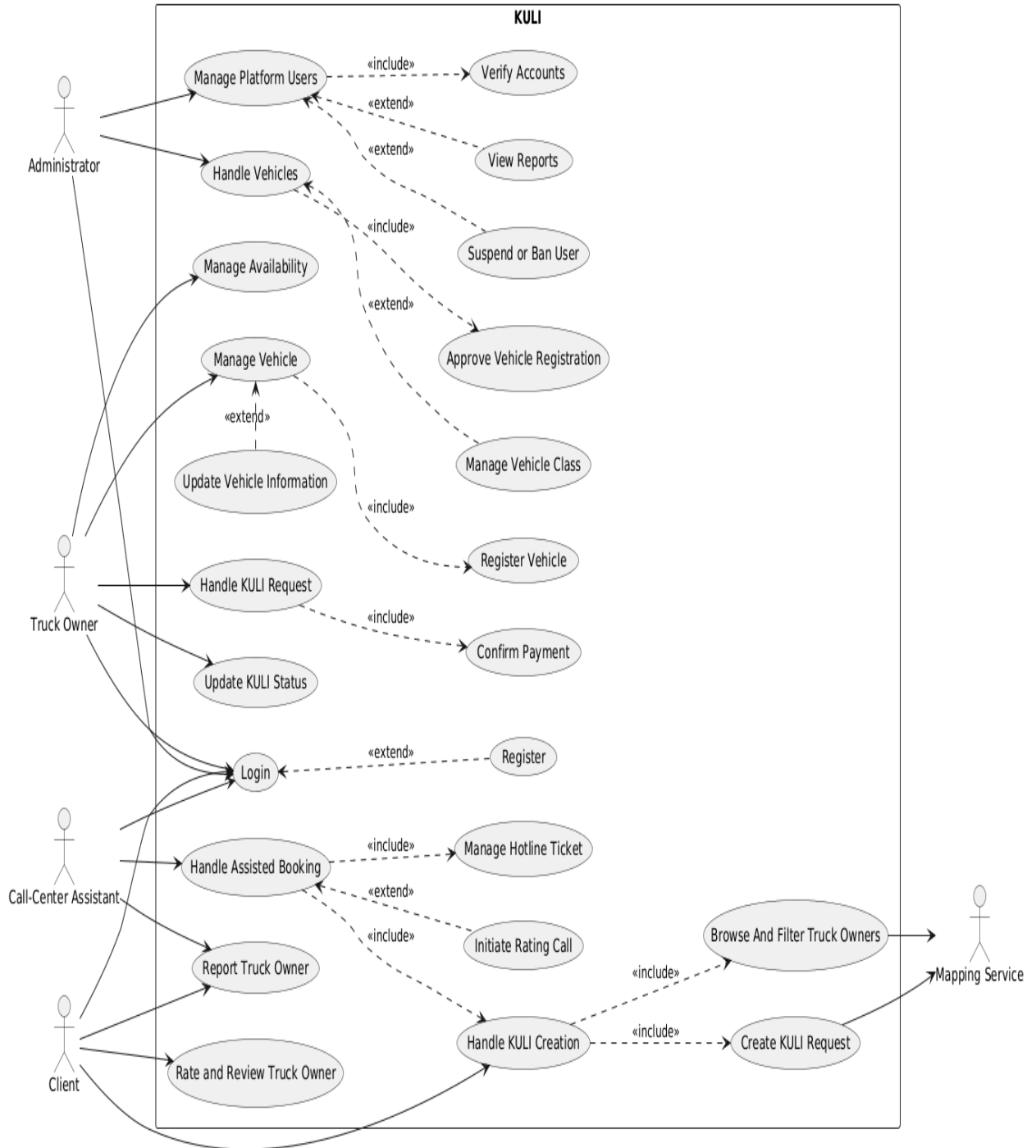


Figure 2.1: Use Case Diagram

2.3.6.2.2 Description Of Use Case Model**2.6.2.2-1: UC-001**

Table 2.7: UC-001

Use Case Name	Login / Register
Corresponding FR	FR-1.1, FR-1.2, FR-1.3, FR-1.4
Description	Allows Clients, Truck Owners, Admins, and Call Assistants to create accounts or log in.
Actors	Client, Truck Owner, Admin, Call Assistant
Pre Conditions	• User has access to the system • For login: account exists • For registration: valid credentials available
Flow of Events	1. Select Login/Register 2. System displays form 3. Actor submits credentials 4. System validates 5. For login → authenticate 6. For registration → store new account 7. Redirect to dashboard
Post Conditions	• User authenticated or registered

	• System maintains session
Alternative Flows	• Invalid credentials → correction • Duplicate data → reject • Reset password option

2.6.2.1-2: UC-002

Table 2.8: UC-002

Use Case Name	Manage Users / Reports
Corresponding FR	FR-2.2, FR-8.4, FR-9.1
Description	Admin reviews reports and manages user accounts.
Actors	Admin
Pre Conditions	• Admin is logged in • Reports or accounts exist
Flow of Events	1. Admin selects management module 2. System shows users/reports 3. Admin selects entry 4. Admin applies action (verify/suspend/ban/edit/remove) 5. System saves changes

Post Conditions	• User state or report status updated
Alternative Flows	• Invalid action blocked • Missing details → request more data

2.6.2.1-3: UC-003

Table 2.9: UC-003

Use Case Name	Register / Manage Vehicle
Corresponding FR	FR-2.1, FR 10.1
Description	Truck Owners register vehicles, set active vehicle, and update details.
Actors	Truck Owner
Pre Conditions	• Truck Owner logged in • Valid documents available
Flow of Events	1. Open Vehicle Management 2. System shows vehicles 3. Submit new vehicle 4. System validates 5. Store vehicle

	6. Owner may set active vehicle or update info
Post Conditions	• Vehicle registered/updated • Active vehicle set
Alternative Flows	• Invalid/missing documents • Duplicate license plate

2.6.2.1-4: UC-004

Table 2.10: UC-004

Use Case Name	Create KULI Request
Corresponding FR	FR-3.1, FR-3.2, FR-4.1, FR-4.3, FR-5.1, FR-7.3
Description	Client or Call Assistant creates a new KULI request.
Actors	Client, Call Assistant
Pre Conditions	• Actor logged in • Pickup/destination data available
Flow of Events	1. Select Create Request 2. Fill trip details 3. Submit request 4. System calculates estimate

	5. Stores request 6. Notifies Truck Owners
Post Conditions	• Request created • Truck Owners alerted
Alternative Flows	• Missing fields • No available vehicles

2.6.2.1-5: UC-005

Table 2.11: UC-005

Use Case Name	Handle KULI Request
Corresponding FR	FR-10.2, FR-7.3, FR-2.3
Description	Truck Owners view, accept, ignore, or cancel KULI requests.
Actors	Truck Owner
Pre Conditions	• Truck Owner logged in • Pending requests exist
Flow of Events	1. Open inbox 2. System shows requests 3. Owner selects one

	4. Accept/ignore/cancel 5. System updates status 6. Notify client if accepted
Post Conditions	● Request status updated
Alternative Flows	● Another owner accepts first ● Invalid action

2.6.2.1-6: UC-006

Table 2.12: UC-006

Use Case Name	Update KULI Status
Description	Truck Owner updates a KULI state during the trip.
Corresponding FR	FR-6.1, FR-6.2, FR-10.3 FR 7.3
Actors	Truck Owner
Pre Conditions	● KULI accepted ● Owner logged in
Flow of Events	1. Open active KULI 2. View current status 3. Update status (in-progress/arrived/completed)

	4. System saves 5. Notify Client
Post Conditions	• Status updated
Alternative Flows	• Invalid status change

2.6.2.1-7: UC-007

Table 2.13: UC-007

Use Case Name	Rate Terminated KULI
Corresponding FR	FR-8.1, FR-8.2
Description	Clients rate a completed or cancelled KULI.
Actors	Client
Pre Conditions	• KULI completed/cancelled • Client logged in
Flow of Events	1. Select KULI 2. System displays rating form 3. Client submits rating 4. System saves 5. Recalculate owner rating

Post Conditions	● Rating stored
Alternative Flows	● Missing rating value

2.6.2.1-8: UC-008

Table 2.14: UC-008

Use Case Name	Report Truck Owner
Corresponding FR	FR-8.4
Description	Client or Assistant submits a misconduct report.
Actors	Client, Call Assistant, Admin
Pre Conditions	● KULI exists ● Actor logged in
Flow of Events	1. Select report option 2. Provide incident details 3. Submit report 4. System saves 5. Admin reviews and acts
Post Conditions	● Report logged ● Admin decision applied

Alternative Flows	• Incomplete report • Invalid report rejected
--------------------------	--

2.6.2.1-10: UC-09

Table 2.16: UC-09

Use Case Name	Handle Vehicles
Description	Admin handles vehicle categories and approvals.
Corresponding FR	FR-2.2, FR-9.1
Actors	Admin
Pre Conditions	• Admin logged in • Vehicle entries exist
Flow of Events	1. Open vehicle management 2. View pending approvals 3. Select vehicle 4. Approve/edit/remove 5. Save changes
Post Conditions	• Vehicle registry updated

Alternative Flows	• Invalid documents • Missing details
--------------------------	--

2.6.2.1-11: UC-010

Table 2.17: UC-010

Use Case Name	Handle Assisted Booking
Description	Call Assistant books a KULI on behalf of a client.
Corresponding FR	FR-7.2, FR-3.1, FR-5.1
Actors	Call Assistant
Pre Conditions	• Assistant logged in • Client info available
Flow of Events	1. Receive call 2. Open assisted booking page 3. Enter client and trip data 4. System searches truck owners and calculates cost 5. Assistant confirms selection with client 6. Request stored and notifications sent
Post Conditions	• Assisted KULI created

Alternative Flows	• Incomplete client data • Request canceled by client
--------------------------	--

2.6.2.1-12: UC-011

Table 2.18: UC-011

Use Case Name	Handle Hotline Ticket
Description	Assistant updates ticket progress during assisted bookings.
Corresponding FR	FR-7.2, FR-6.1
Actors	Call Assistant
Pre Conditions	• Ticket exists • Assistant logged in
Flow of Events	1. Open ticket dashboard 2. Select ticket 3. Update status 4. System saves 5. Notify Client
Post Conditions	• Ticket status updated
Alternative Flows	• Closed ticket cannot be edited

2.6.2.1-13 UC-012

Table 2.19: UC-012

Use Case Name	Confirm Payment
Description	Truck Owner confirms payment after KULI completion.
Corresponding FR	FR-5.2, FR-5.3, FR-7.3
Actors	Truck Owner
Pre Conditions	● KULI completed ● Truck Owner logged in
Flow of Events	1. Select completed KULI 2. View cost 3. Confirm payment 4. System records it 5. Client notified
Post Conditions	● Payment status updated
Alternative Flows	● Disputed amount → notify Admin

2.6.3 Object Model

2.6.3.1 Data Dictionary

Table 2.20: Data Dictionary

Object / Entity	Type	Attributes / Methods	Description
User	Entity	userID, role, name, email, phone, password, address	Represents a user involved in the system
Role	Entity	roleID, name	Represents user role
Vehicle	Entity	vehicleID, ownerID, type, capacity, licensePlate, status, insurance, documents	Represents a truck registered in the system
KULI	Entity	requestID, clientID, ownerID, pickupLocation, pickupTime, destination, loadDetails, status, tip, Cost	Represents a transportation request
Ticket	Entity	ticketID, requestID, assignedAssistant, status	Represents a hotline ticket when no Truck Owner is immediately available
Report	Entity	reportID, reporterID, truckOwnerID, description, status	Represents complaints about Truck Owners or system issues

Location	Entity	locationID, latitude, longitude, address	Represents geospatial data for pickup and destination
Notification	Entity	notificationID, userID, message	Represents notifications sent to users

2.6.3.2 Class Modeling

KULLI: P2P Logistics Mobile Application On Demand Trucking Service

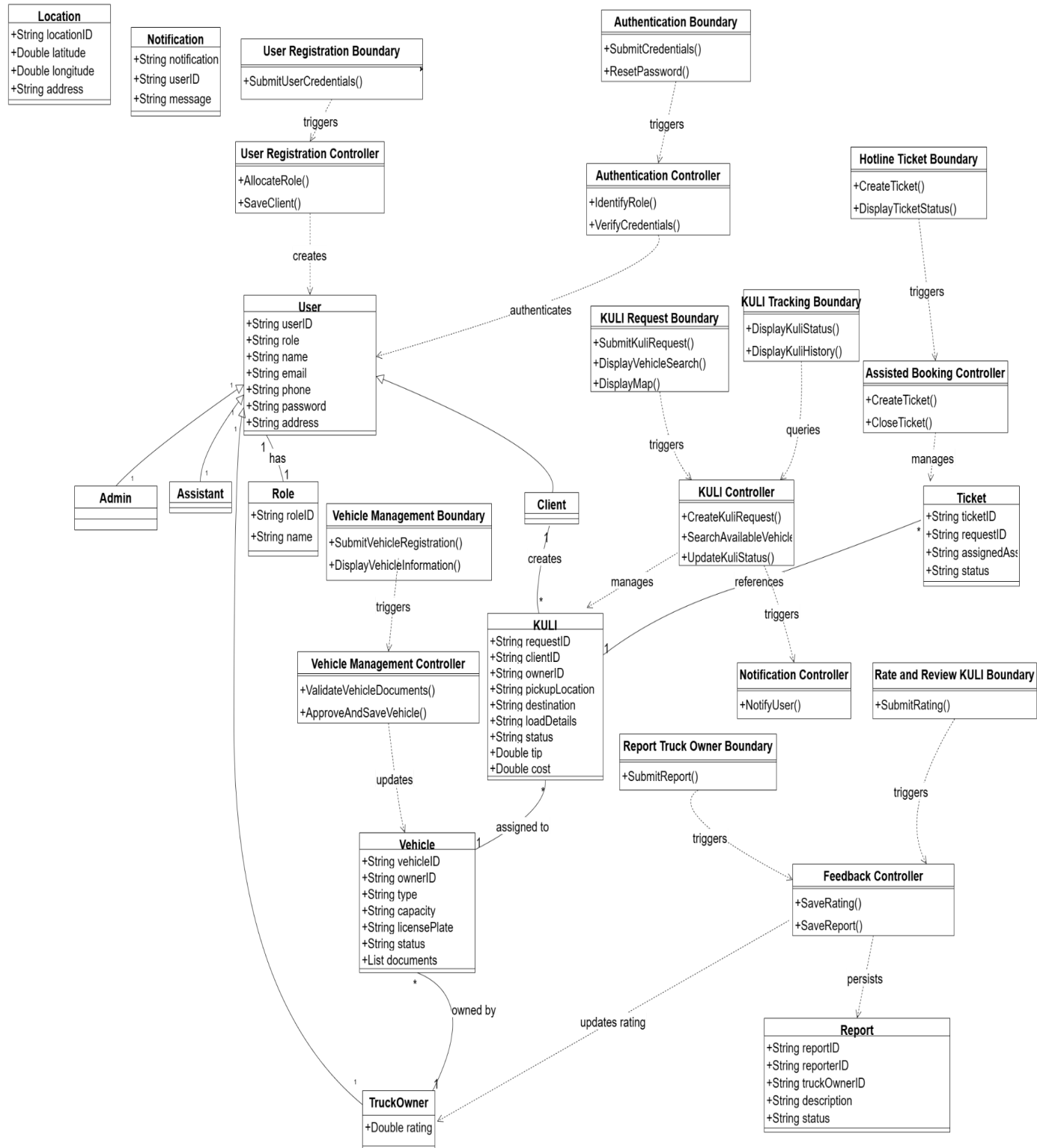


Figure 2.2: Class Model diagram

2.3.6.4 Dynamic Modeling

This section will illustrate the dynamic model of the system with sequence diagrams, activity charts, and state charts.

2.3.6.4.1 Sequence Diagrams

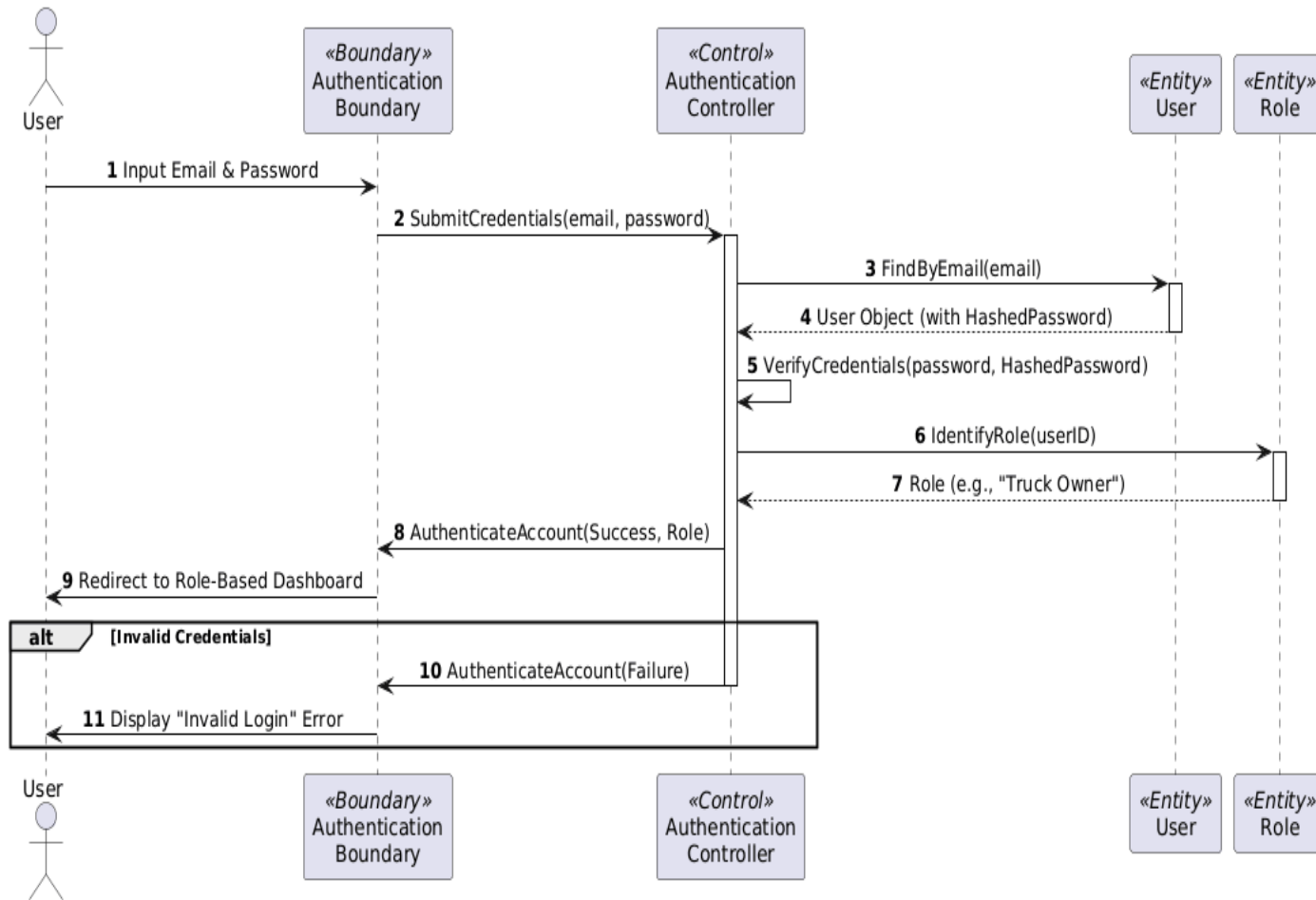


Figure 2.3: Authentication Diagram

KULLI: P2P Logistics Mobile Application On Demand Trucking Service

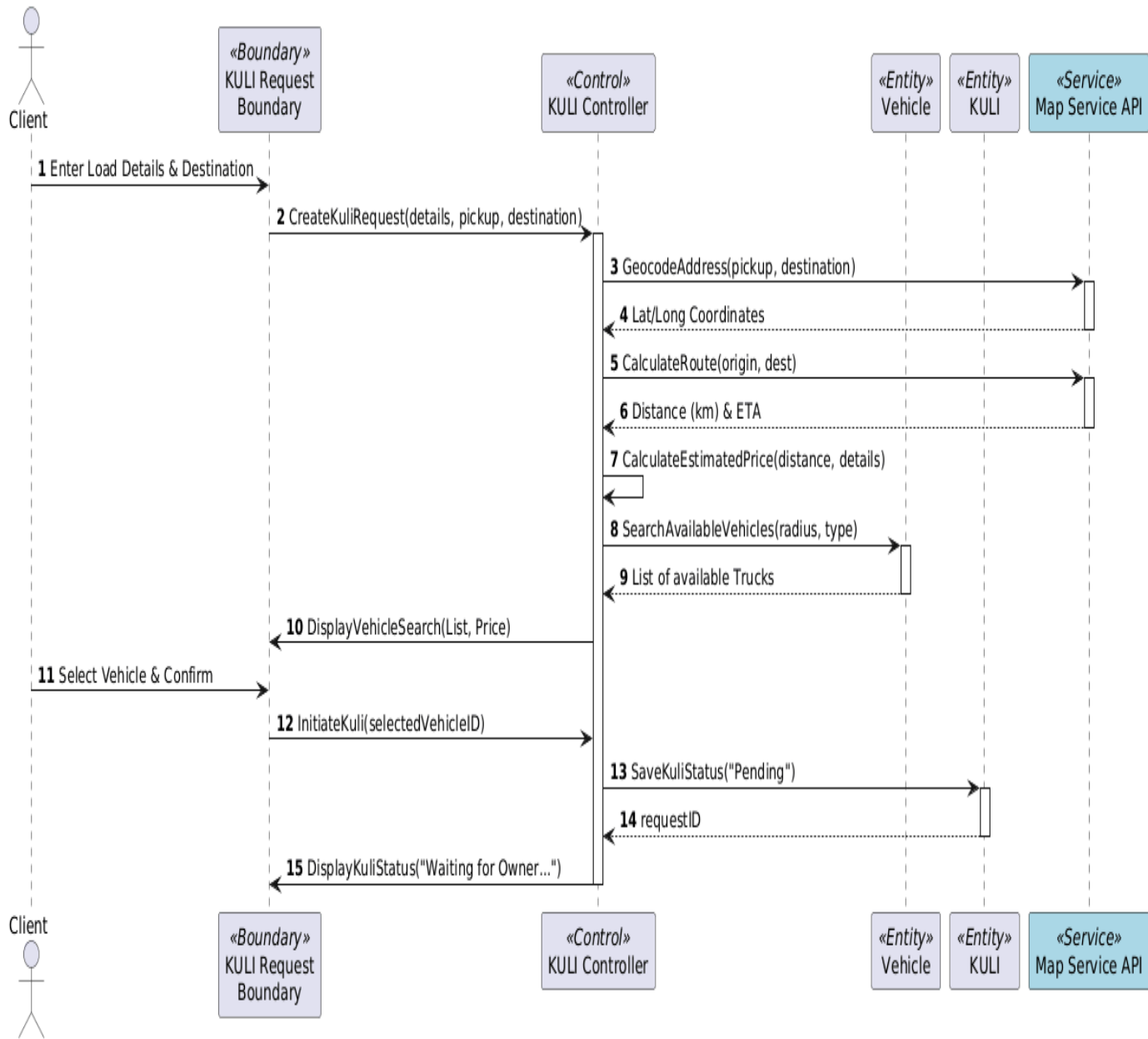


Figure 2.4: Kuli Request

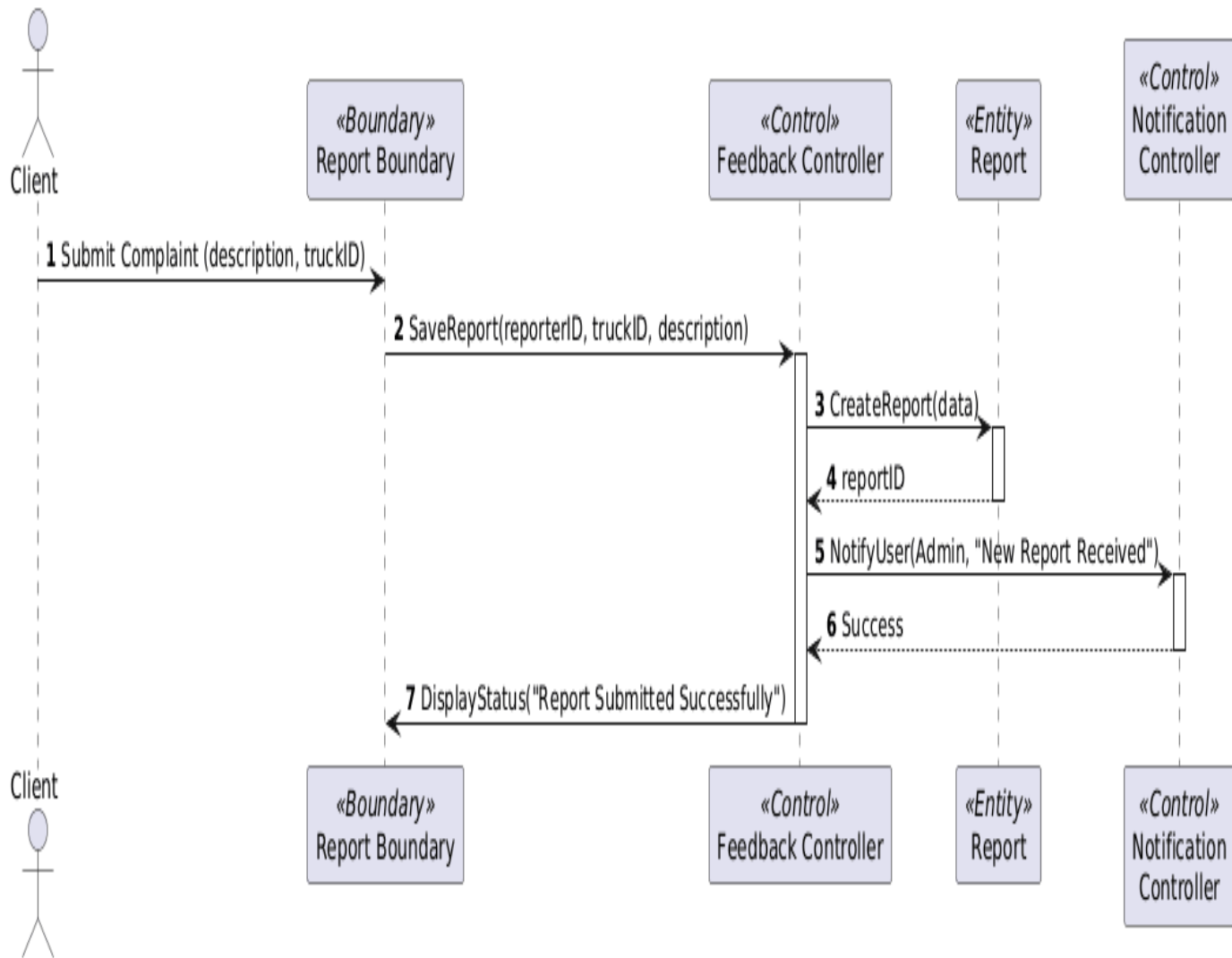


Figure 2.5: Report Truck Owner

KULLI: P2P Logistics Mobile Application On Demand Trucking Service

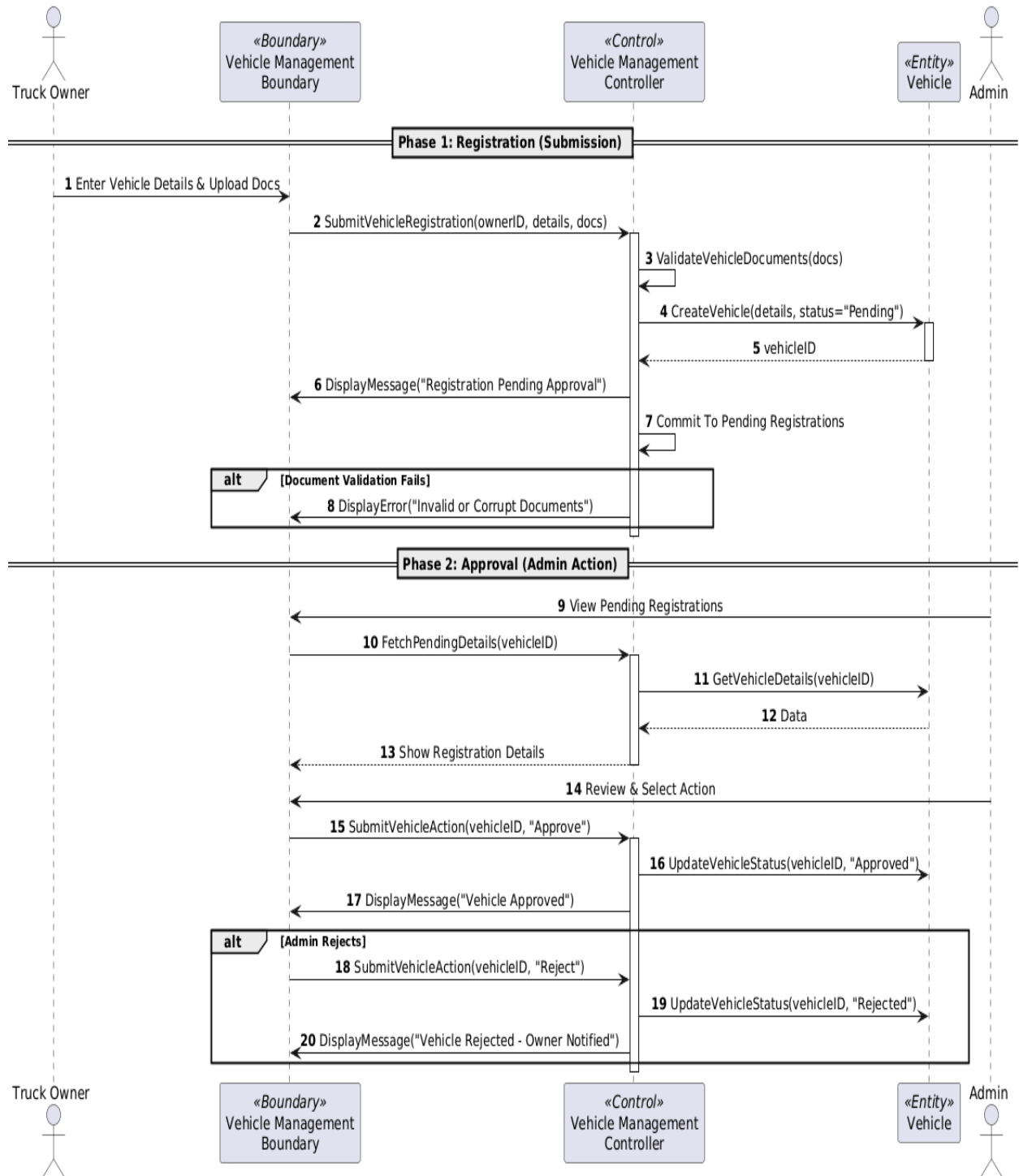


Figure 2.6: Vehicle Approval

2.6.4.2 Activity Diagrams

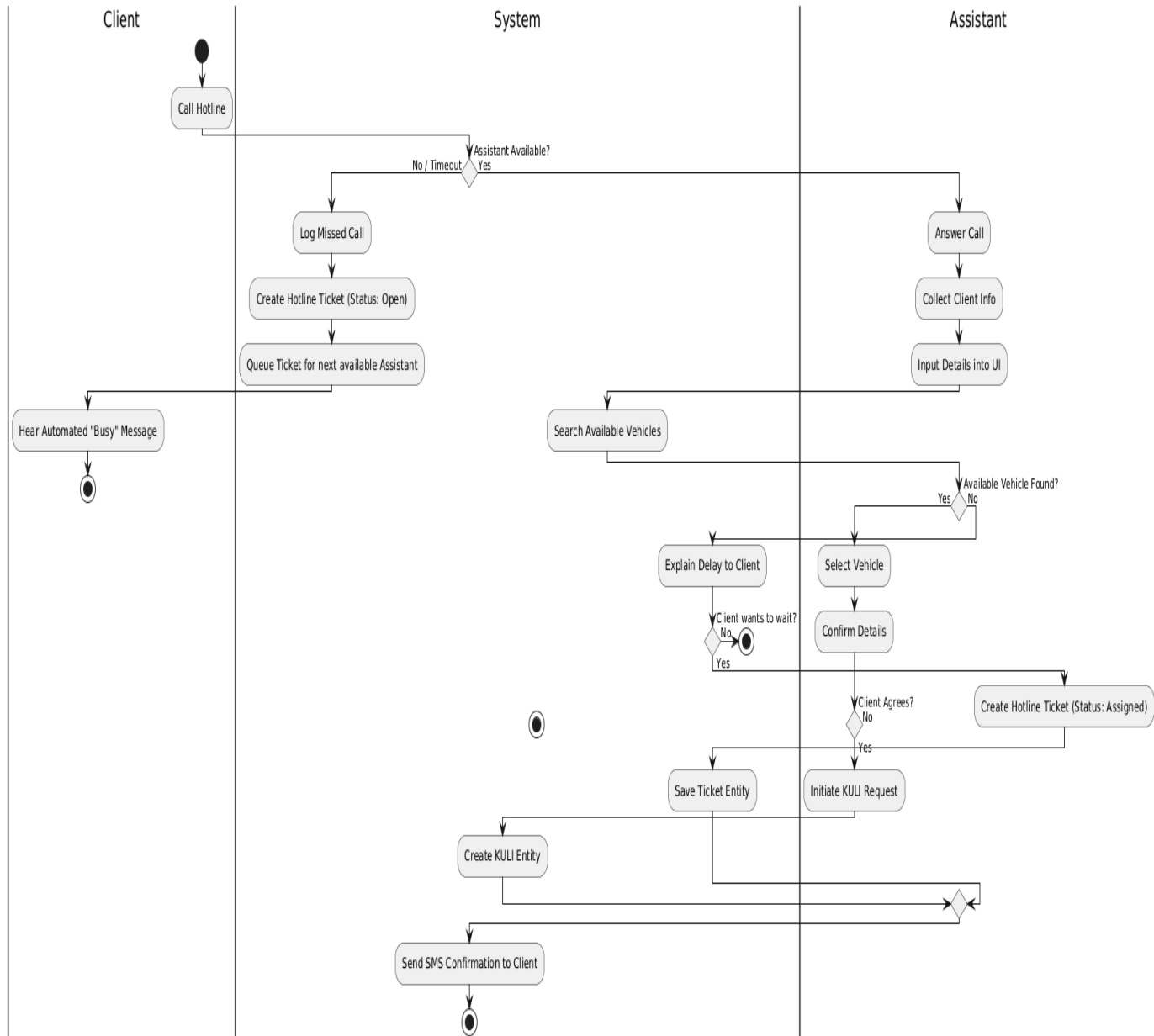


Figure 2.7: Assisted Booking

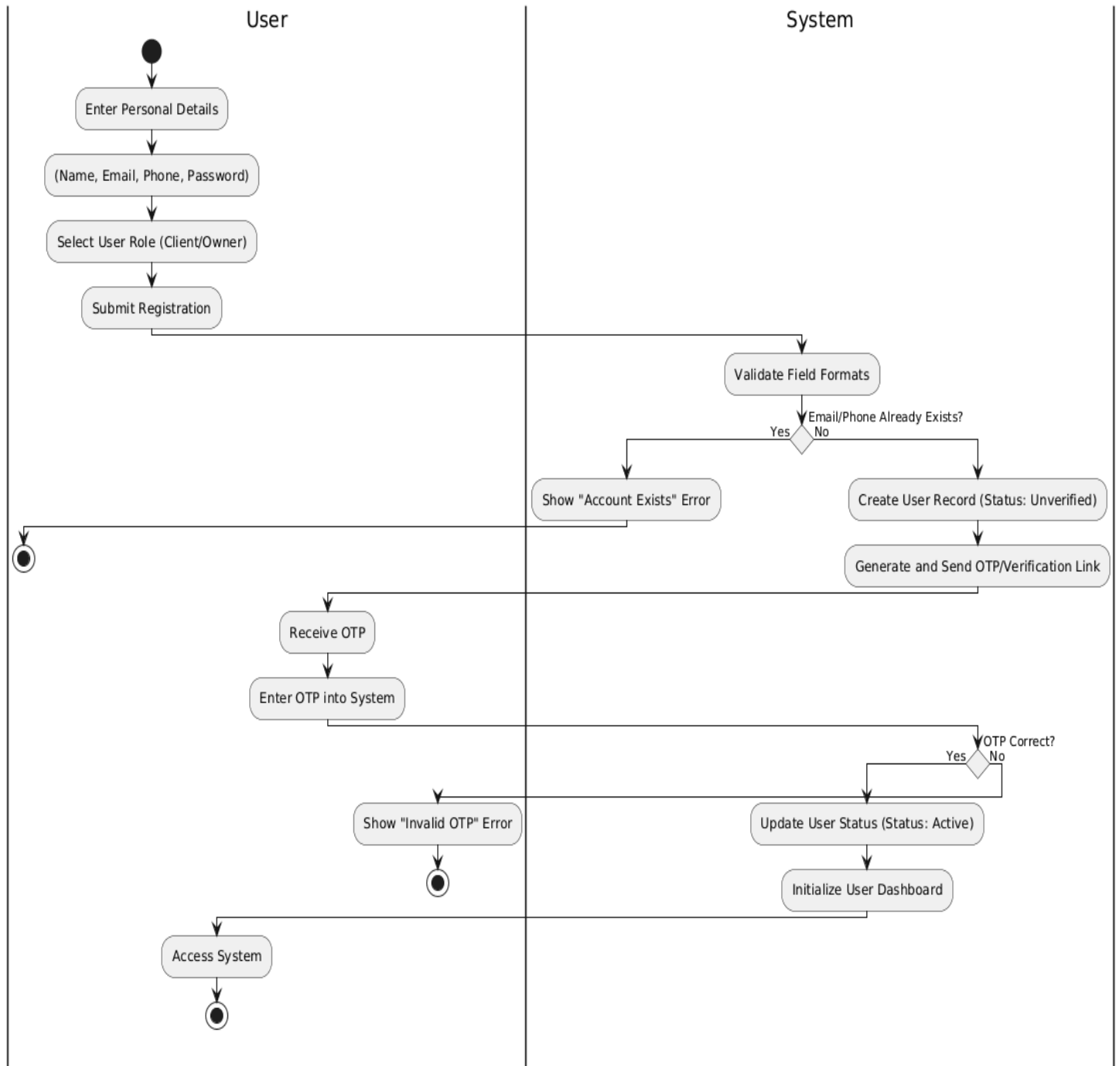


Figure 2.8: Registration

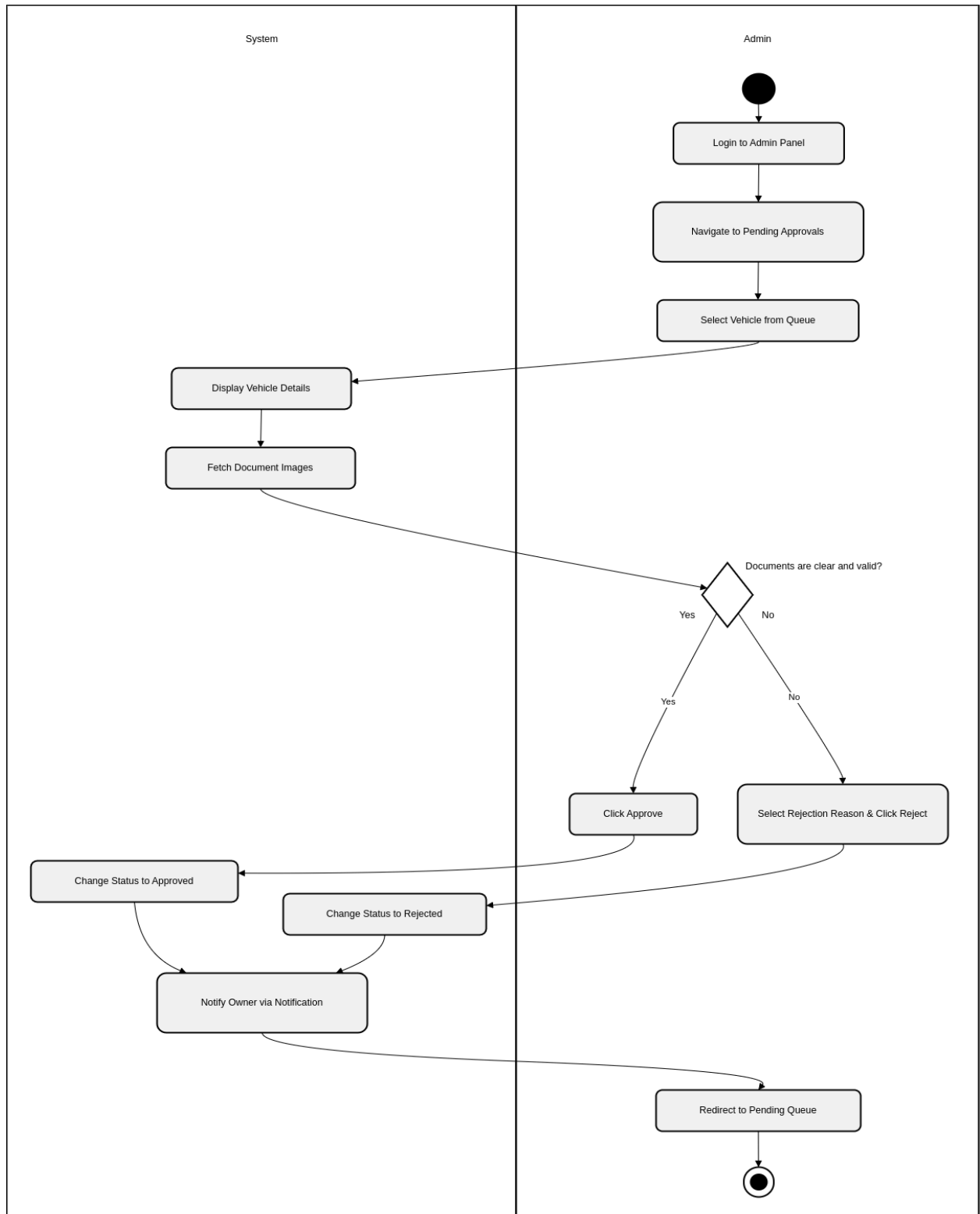


Figure 2.9: Vehicle Approval

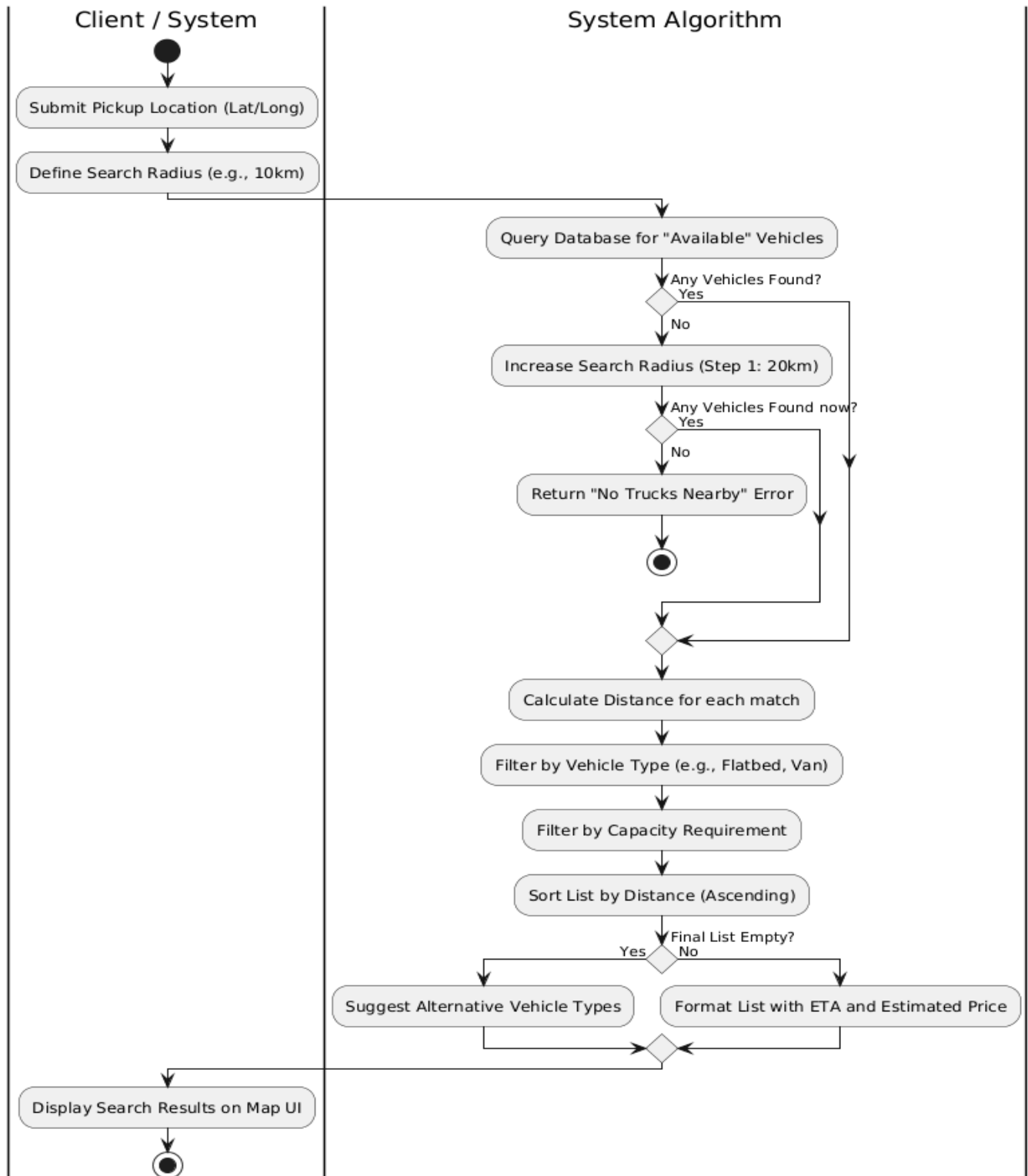


Figure 2.10: Search Nearby Vehicle

2.6.4.3 State Charts

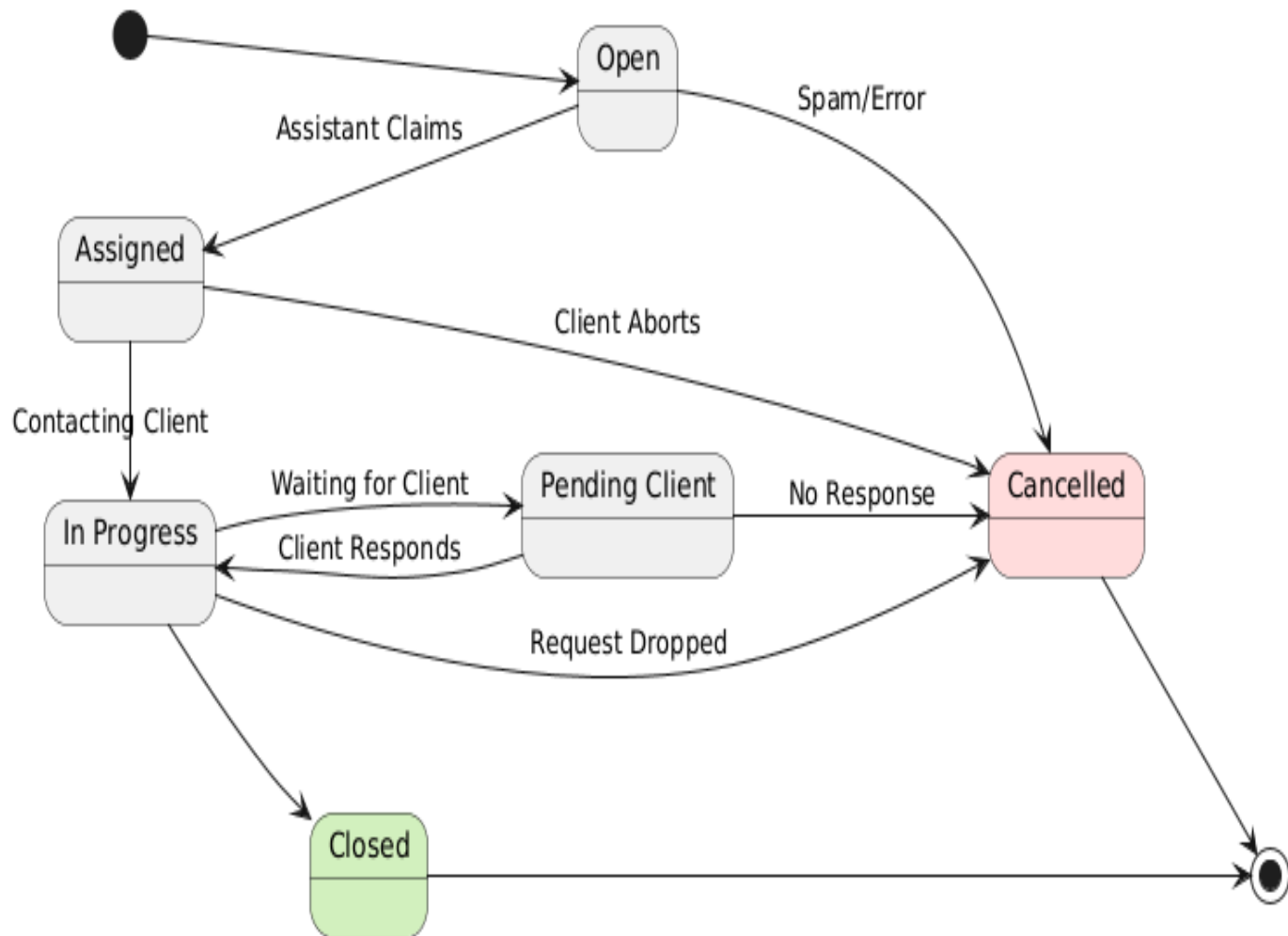


Figure 2.11: Ticket State Chart

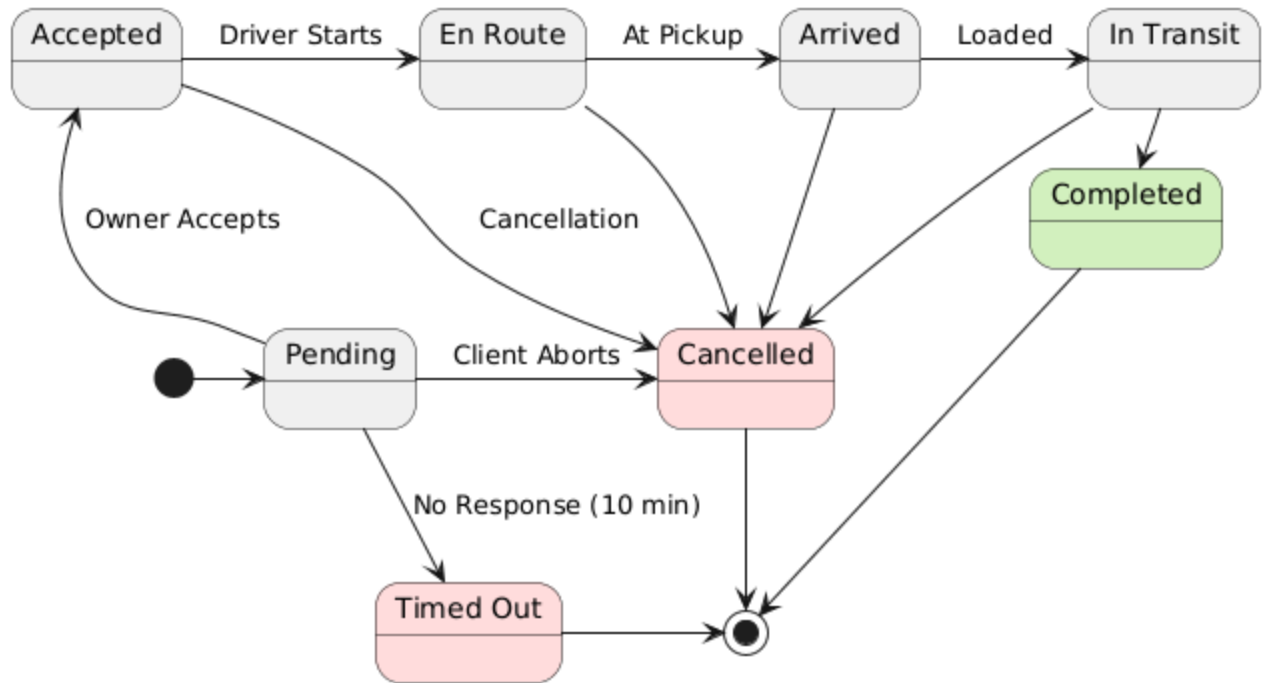


Figure 2.12: Kuli State Chart

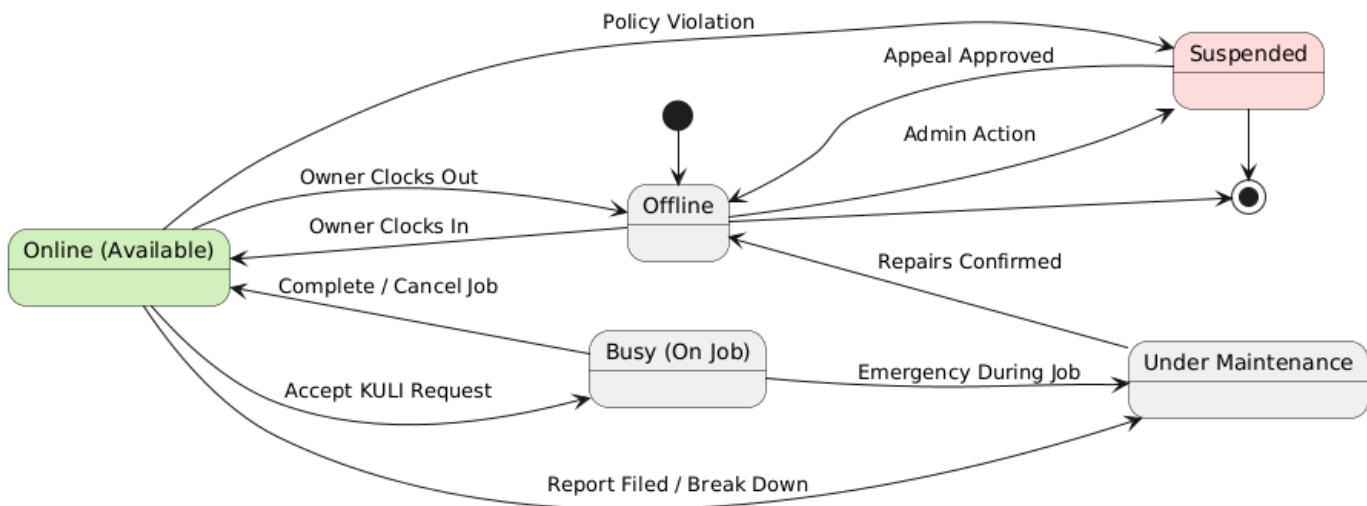


Figure 2.13: Vehicle State Chart

2.6.5 Client Interface

Overview of UI style and direction

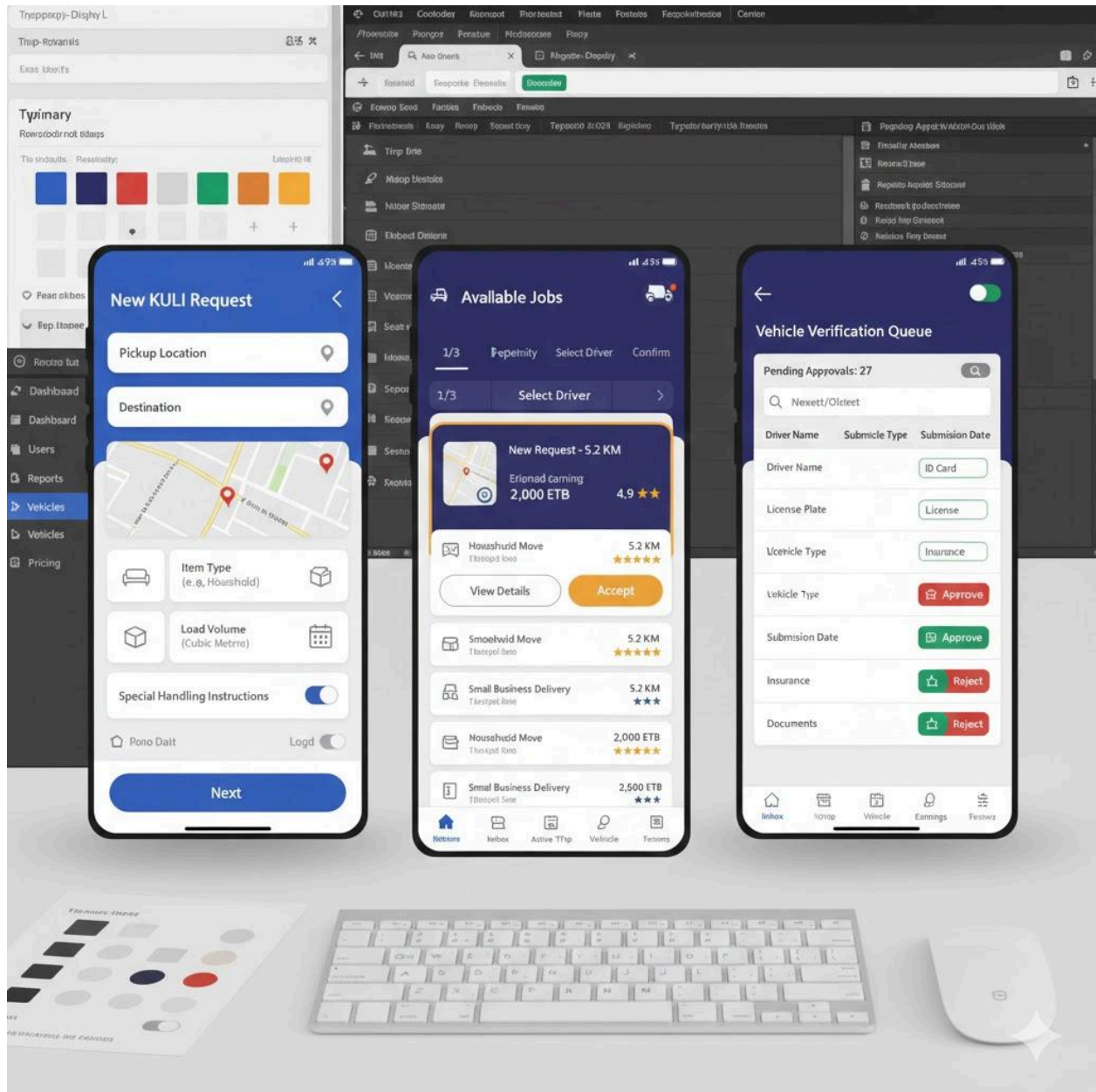


Figure 2.14: UI Template

3. SYSTEM DESIGN

3.1. Introduction

This chapter defines the technical transition from requirement analysis to system implementation for the **Kuli P2P Logistics Platform**. The design translates the functional needs—such as on-demand truck booking and assisted hotline services—into a structured, three-tier architecture.

Traceability & Constraints: The design is directly traced to the requirements identified in Chapter 2, specifically addressing the need for proximity-based matching and multi-role access. Key constraints impacting this architecture include the need for a low-cost initial deployment and the exclusion of real-time GPS/Payment gateways in favor of a manual status-update workflow.

Design Strategy Overview:

- **Subsystem Decomposition:** The backend is organized into 8 domain-specific modules (IAM, UA, TR, BP, LO, PM, FD, SU) to ensure high maintainability.
- **Hardware/Software Mapping:** Use of standard smartphones and workstations connecting to a centralized Node.js application server.
- **Persistent Data Management:** Utilization of MongoDB for flexible document storage and geospatial indexing.
- **Access Control & Security:** Implementation of Role-Based Access Control (RBAC) managed by the Identity & Access module.

3.2. Current Software Architecture

The system being replaced is a **manual, informal logistics network**. Currently, truck owners and clients rely on word-of-mouth, physical bulletin boards, and unmanaged phone calls.

- **Architecture Type:** Ad-hoc / Decentralized.
- **Limitations:** This "architecture" lacks a centralized data repository, leading to non-transparent pricing, lack of vehicle verification, and no formal record-keeping. The proposed system replaces this fragmented workflow with a unified digital engine.

3.3. Proposed Software Architecture

3.3.1. Overview

The proposed architecture for the Kuli platform is a **Layered Modular Monolith**. This bird's-eye view shows a three-tier structure comprising a Client Tier (Mobile and Web), an Application Tier (Node.js Backend), and a Data Tier (MongoDB and Archival Storage).

Assignment of Functionality: The core business logic is encapsulated within the **Application Tier** and distributed across the following subsystems:

- **Identity & Access (IAM):** Secures the system via authentication and RBAC.
- **User Accounts (UA):** Manages profiles for Clients, Owners, and Staff.
- **Truck Registry (TR):** Handles vehicle enrollment and compliance documents.
- **Booking & Pricing (BP):** Manages the marketplace, pricing, and truck discovery.
- **Live Operations (LO):** Orchestrates active trip states and manual status logging.
- **Financial Settlement (PM):** Generates invoices and tracks cash-based transactions.
- **Feedback & Disputes (FD):** Manages ratings and administrative mediation.
- **System Utilities (SU):** Provides notification dispatch and audit logging.

This modularity ensures that while the system is deployed as a single unit for simplicity, it remains **scalable by design**, allowing individual modules to be extracted or expanded as the platform grows.

Architectural Style and Rationale

The chosen architectural style is the **Modular Monolith**. This style was selected over a traditional "Big Ball of Mud" monolith or a complex "Microservices" architecture for the following reasons:

- **Rationale for Modularity:** Unlike a standard monolith where all code is tightly coupled, a modular monolith enforces boundaries. This allows the team to work on the "Financial Settlement" module without risking side effects in the "Live Operations" module, satisfying the need for **system reliability**.

- **Justification for the Monolithic Deployment:** At this stage of the project, microservices would introduce unnecessary network latency and DevOps complexity (orchestration, service discovery, etc.). A single deployment unit simplifies testing and reduces infrastructure costs while the product-market fit is being established.
- **Future-Proofing (Scalability):** Because the subsystems are logically separated and communicate through defined internal APIs, the architecture provides a clear path to scalability. If a specific module (like Booking & Pricing) becomes a bottleneck, it can be extracted into a standalone service with minimal refactoring.
- **Consistency:** A unified data tier (MongoDB) ensures strong data consistency for logistics transactions, which is harder to achieve in distributed architectural styles.

System Scope & Constraints

To maintain a lightweight core, the architecture focuses on essential coordination. Real-time GPS telemetry and third-party payment gateways are excluded from the current scope. The modular structure ensures these can be added as external service integrations in future versions without altering the base architecture.

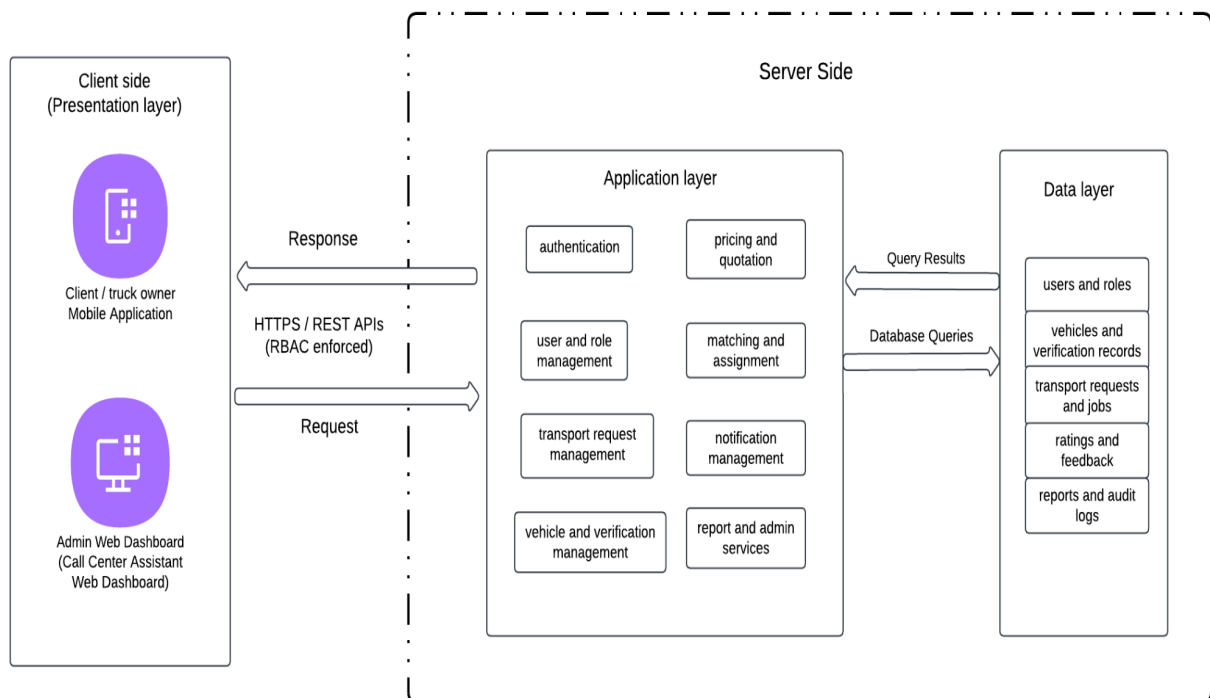


Figure 3.1: Architectural design diagram

3.3.2. Subsystem Decomposition

The KULI system is composed of the following major subsystems, each having dedicated responsibilities:

- 1. Access & Gateway (AG):** Serves as the system's entry point, managing request routing, API gateway logic, protocol transformation, rate limiting, and cross-platform interface normalization.
- 2. Identity Provider (ID):** Handles core security protocols, including credential management, multi-factor authentication (MFA) via OTP, secure session token issuance, and role-based access control (RBAC) enforcement.
- 3. User Accounts (UA):** Manages the lifecycle of user digital personas, including multi-role profile metadata, saved preferences, billing/payout account details, and administrative account oversight.
- 4. Truck Registry (TR):** Orchestrates the supply-side ecosystem, managing vehicle enrollment, document archival for compliance, administrative verification workflows, and real-time fleet availability status.
- 5. Booking & Pricing (BP):** Functions as the marketplace engine, handling logistics request validation, algorithmic pricing based on distance and vehicle class, and proximity-based truck discovery and matching.
- 6. Live Operations (LO):** Manages active trip states through a synchronized state machine, facilitating real-time GPS telemetry, in-transit milestone logging, and context-bound messaging between participants.
- 7. Financial Settlement (PM):** Governs the fiscal integrity of the system, managing transaction recording, automated invoice generation, payout scheduling for owners, and tamper-proof financial auditing.
- 8. Feedback & Disputes (FD):** Oversees platform quality and integrity, managing multi-criteria rating aggregation, text review moderation, dispute filing, and administrative mediation workflows.
- 9. System Utilities (SU):** Provides cross-cutting infrastructure services, including a centralized notification dispatch hub (SMS/Email/Push) and a system-wide audit logging service for compliance and health monitoring.

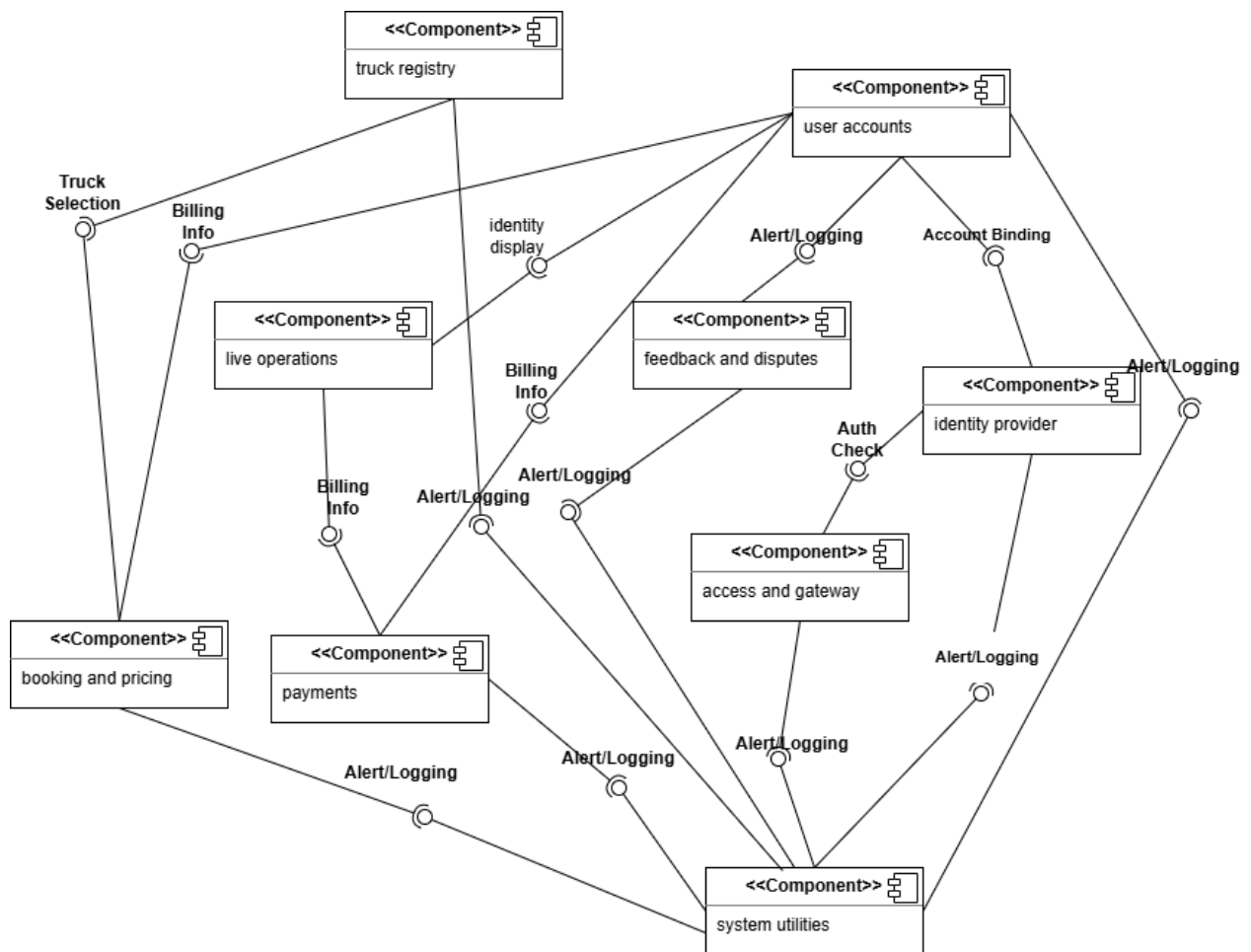


Figure 3.2: Component Diagram

3.3.3. Hardware-Software Mapping

The **Kuli** platform follows a client-server model built on a **Node.js modular monolith**. Users interact with the system via a **React Native mobile app** (Android/iOS), while administrators utilize a **React-based web dashboard** on standard workstations. The backend leverages **REST/JSON** for transactional logic and in the future (later versions), **WebSockets** for real-time **GPS telemetry** within the Live Operations module. Data is persisted in **MongoDB** using **geospatial indexing** for efficient truck discovery. To maintain a lightweight core, the architecture integrates **off-the-shelf APIs** for mapping and routing and **Supabase** for authentication.

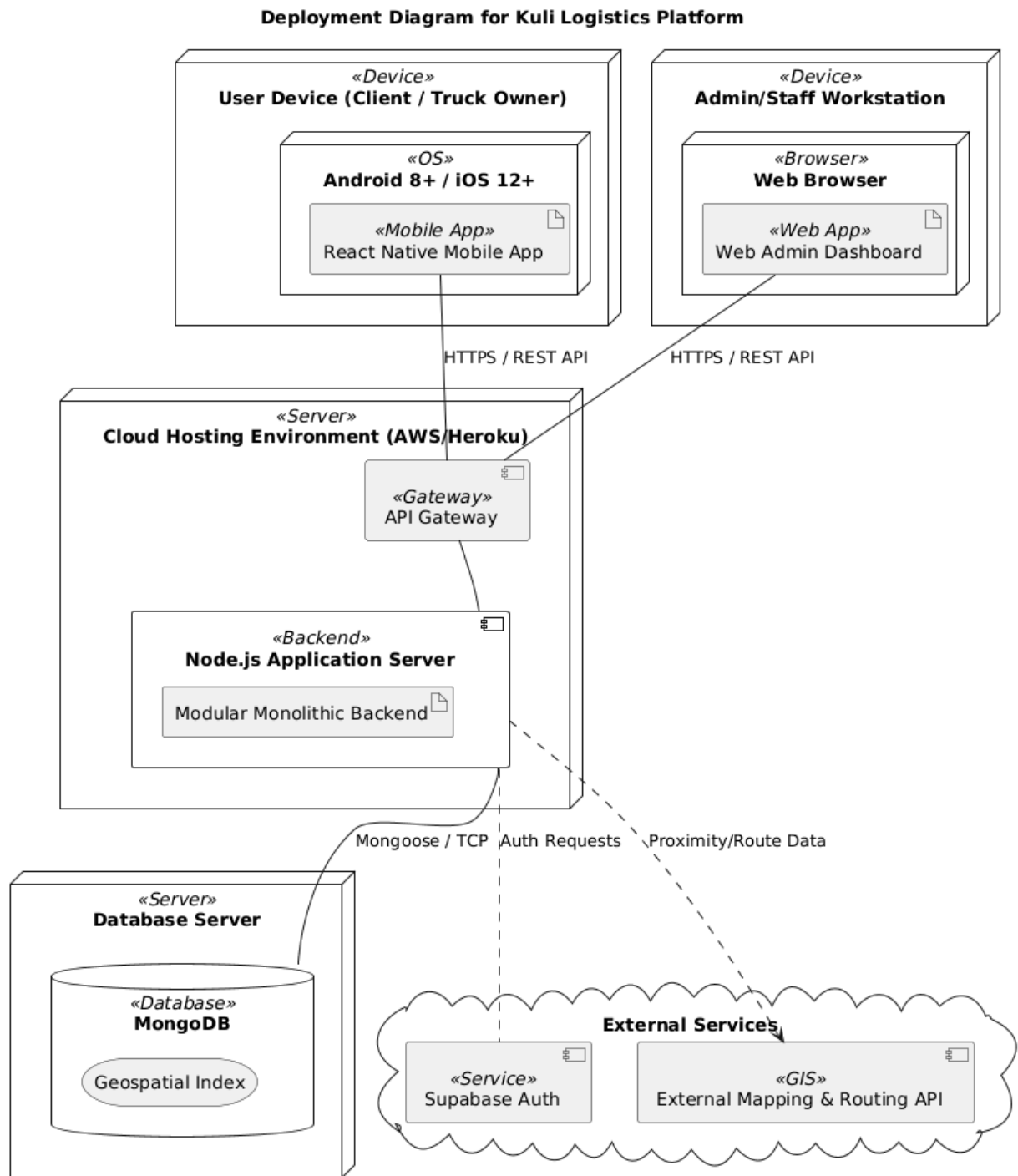


Figure 3.3: Deployment Diagram

3.3.4. Persistent Data Management

3.3.4.1. Description of Data Schemes

Based on the system's core entities, the data model is organized into several key document collections:

- **Users:** Stores digital personas for all roles (Client, Truck Owner, Assistant, Admin), including authentication credentials, contact details, and role-based metadata.
- **Vehicles:** Manages the truck registry, linking vehicles to their owners and storing compliance data such as license plates, capacity, and insurance documents.
- **Kuli Requests (Trips):** The central transactional entity that tracks the lifecycle of a logistics request from creation to completion, linking clients to truck owners.
- **Locations:** Stores geospatial coordinates (latitude and longitude) for pickup and destination points to facilitate distance calculations and proximity matching.
- **Hotline Tickets:** Manages the state of assisted bookings created by call-center assistants on behalf of users.
- **Reports & Notifications:** Tracks user feedback, disputes, and the history of system-generated alerts.

3.3.4.2. Selection of Database

The platform utilizes **MongoDB** as its primary persistent storage engine. This choice is driven by the following factors:

- **Geospatial Indexing:** MongoDB's native support for **2dsphere** indexes is critical for the "proximity-based listing mechanism," allowing the system to efficiently find available truck owners near a client's pickup location.
- **Schema Flexibility:** As a document-oriented database, MongoDB allows for evolving data structures, which is essential for handling diverse vehicle types and future integration of digital payment gateways.
- **Scalability:** Its distributed nature supports the project's long-term goal of scaling beyond the initial Addis Ababa scope.

3.3.4.3. Description of Database Encapsulation

The database is encapsulated using an **Object-Document Mapper (ODM)**, specifically **Mongoose**, within the Node.js backend. This provides:

- **Data Validation:** Enforcing schema rules at the application level to ensure that required fields like **licensePlate** or **pickupLocation** are present before persistence.
- **Abstraction:** Subsystems interact with high-level JavaScript objects rather than writing raw database queries, which maintains the integrity of the modular monolith architecture.
- **Security:** Direct database access is restricted; all data operations must pass through the Backend Engine, which enforces **Role-Based Access Control (RBAC)** as managed by the Identity & Access (IAM) subsystem.

3.3.4.3. Object Diagram

Since the Kuli platform utilizes a document-oriented database (MongoDB), the following object diagrams serve as representational snapshots of the system's persistent data state.

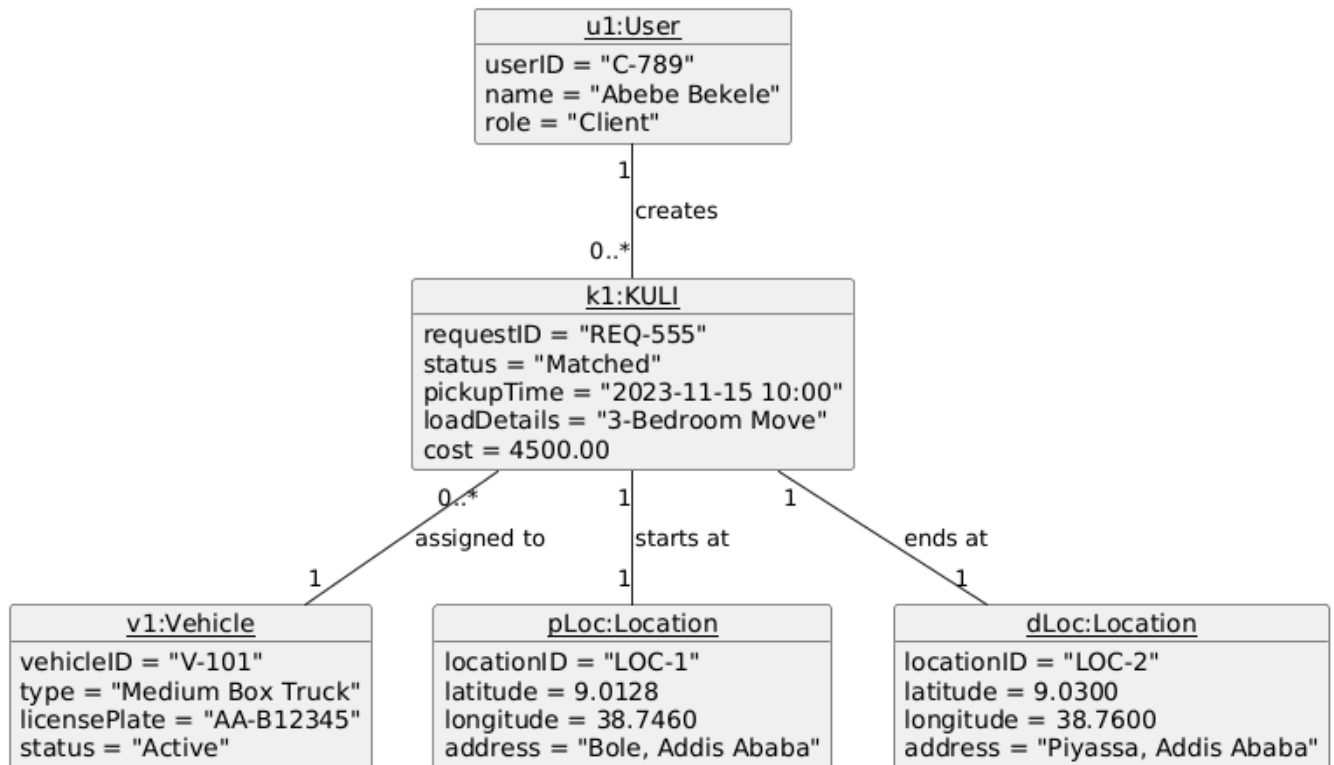


Figure 3.4: Snapshot of a Matched Service Diagram

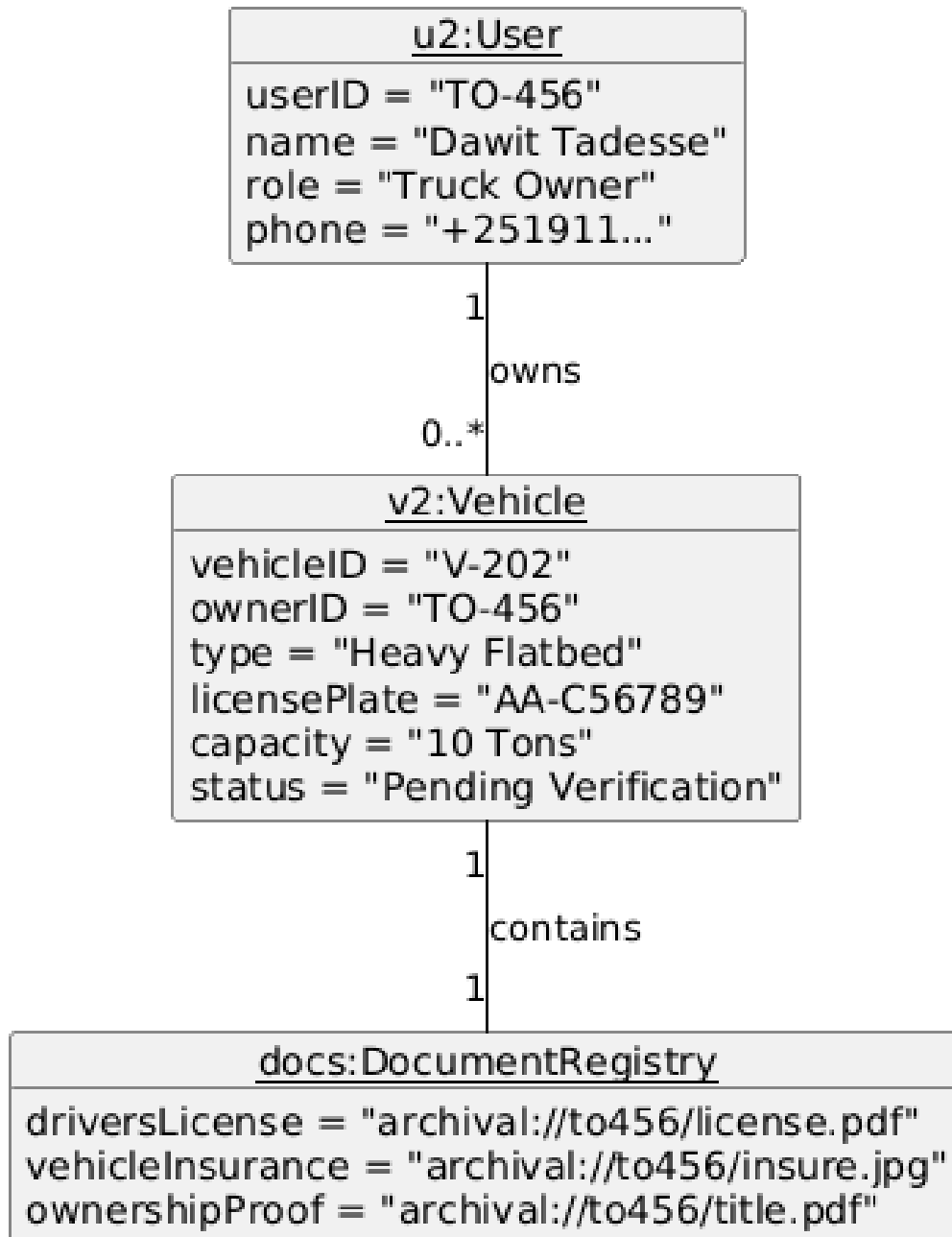


Figure 3.5: Snapshot of Vehicle Onboarding Diagram

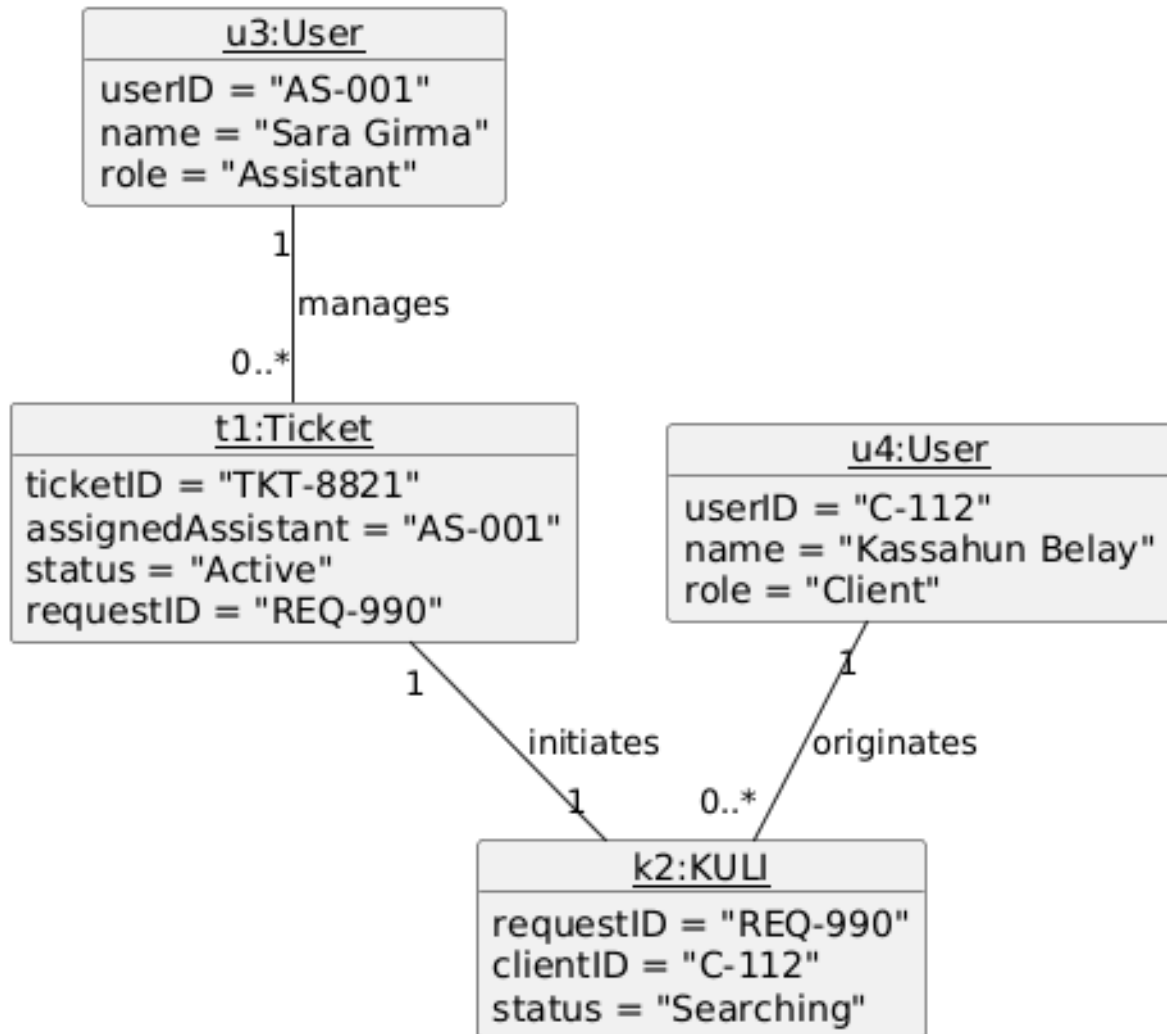


Figure 3.6: Snapshot of Assisted Booking Diagram

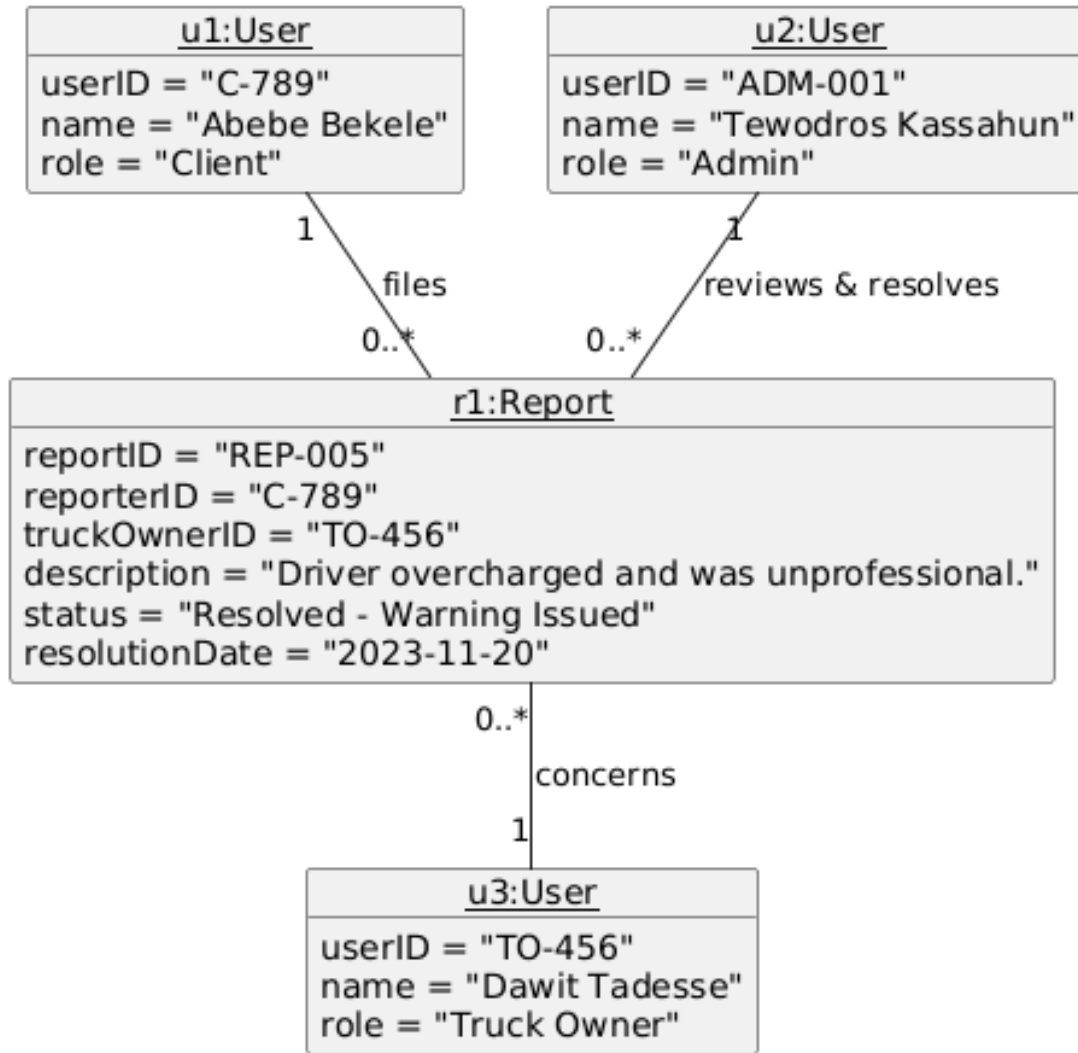


Figure 3.7: Accountability, Trust & Dispute Resolution Diagram

3.3.5. Access Control and Security

The Kuli system utilizes a Role-Based Access Control (RBAC) model to manage system resources and user privacy. Access privileges are categorized by the four primary user roles defined in the system: **Admin**, **Truck Owner**, **Client**, and **Call-Center Assistant**.

Table 3.1: Access Control List (ACL) Table

Packages	Subsystems	Operations	Admin	Client	Truck Owner	Assistant
Edge & Security	Identity Manager	login() / logout()	X	X	X	X
		verifyOTP()	X	X	X	X
		resetPassword()	X	X	X	X
Account Management	User Profile	updateProfile()	X	X	X	X
		suspendAccount()	X			
	Vehicle Registry	registerVehicle()			X	
		verifyVehicle()	X			
Logistics Engine	Booking Manager	createRequest()		X		X
		cancelRequest()		X		X
	Trip Controller	acceptRequest()			X	
		updateTripStatus()			X	
		sendMessage()		X	X	
Settlement & Quality	Finance Manager	confirmPayment()			X	

KULLI: P2P Logistics Mobile Application On Demand Trucking Service

		viewTransactions()	X	X	X	
	Quality Control	submitRating()		X		X
		resolveDispute()	X			
System Utilities	Audit & Logs	viewAuditLogs()	X			
	Notifications	configureAlerts()	X	X	X	X

Security Issues and Mechanisms

- Authentication Mechanism:** The system uses **Supabase Auth** to handle identity management. Users are authenticated via OTP/email or OAuth2 providers. Upon successful login, the system issues JSON Web Tokens (JWT) for session management, utilizing short-lived access tokens (15–60 minutes) and secure refresh tokens to maintain session integrity.
- Encryption:** To ensure data confidentiality, the system employs **TLS (HTTPS)** for all data in transit between the client and backend. For data at rest, sensitive fields (such as National IDs) and database volumes are encrypted using industry-standard algorithms (e.g., AES-256). Passwords are never stored in plain text and are secured using salted hashes (bcrypt/Argon2).
- Key Management:** Cryptographic keys, API secrets, and database credentials are managed via a dedicated cloud **Key Management Service (KMS)** or secrets manager (e.g., Vault). Access to these keys is restricted strictly by role, and keys are rotated periodically or upon suspicion of compromise to minimize security risks.

3.3.6. Subsystem Services

This section outlines the specific services provided by each logical subsystem within the Kuli

Backend Engine. Each service is designed to be autonomous yet collaborative, ensuring a robust and maintainable architecture.

- **Identity & Access (IAM):** Handles user authentication, session management, and Role-Based Access Control (RBAC) to ensure secure access for Clients, Truck Owners, Assistants, and Admins.
- **User Accounts (UA):** Manages the lifecycle of user profiles, including contact information, preferences, and the association of users with specific system roles.
- **Truck Registry (TR):** Orchestrates the supply-side data, managing vehicle enrollment, document storage for compliance, and the verification status of the fleet.
- **Booking & Pricing (BP):** Functions as the marketplace coordinator, validating logistics requests, calculating costs based on distance/class, and performing proximity-based truck matching.
- **Live Operations (LO):** Tracks the active state of logistics trips, managing status transitions (e.g., *Pending* to *In-Transit*) and maintaining the context for communication between participants.
- **Financial Settlement (PM):** Governs the fiscal records of the platform, including transaction logging, manual invoice generation for cash-based workflows, and audit trail maintenance.
- **Feedback & Disputes (FD):** Oversees platform quality by managing user ratings, text reviews, and the formal mediation process for reports handled by Administrators.
- **System Utilities (SU):** Provides cross-cutting infrastructure services, primarily focusing on the centralized notification dispatch hub (SMS/Email/Push) and system-wide audit logging.

3.4.Detailed Class Diagram

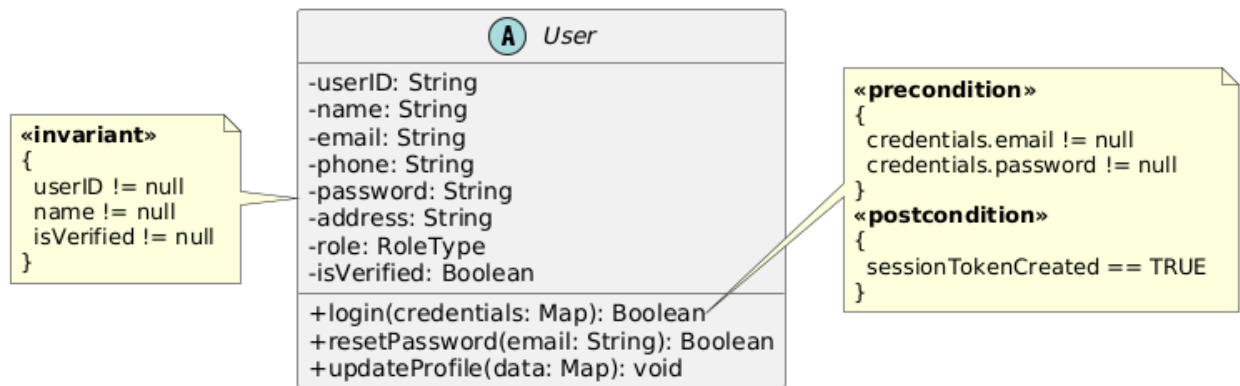


Figure 2.1.4.1. User (Base Class) detailed class diagram

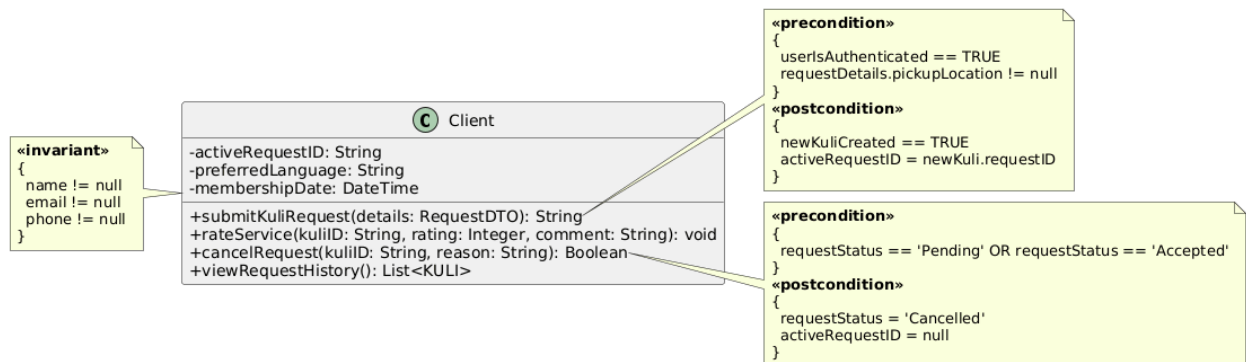


Figure 3.8: Client Entity detailed class diagram

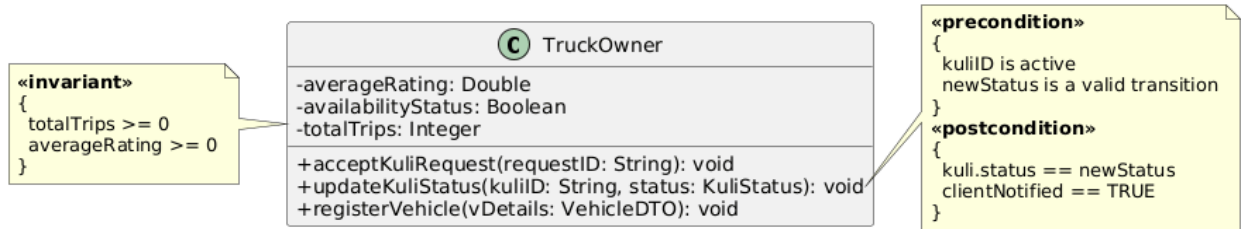


Figure 3.9 Truck Owner Entity

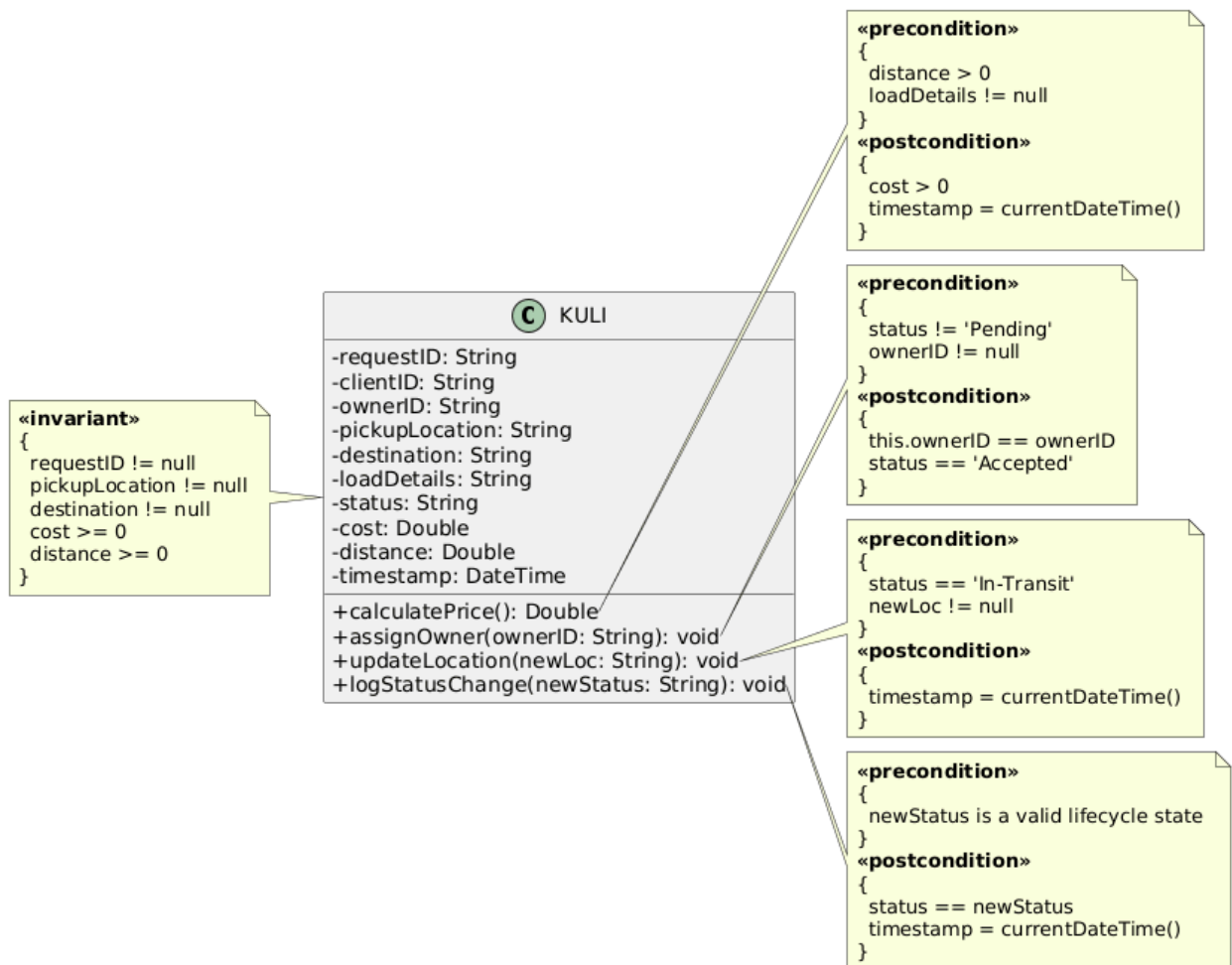


Figure 3.10: KULI (Logistics Request) Entity

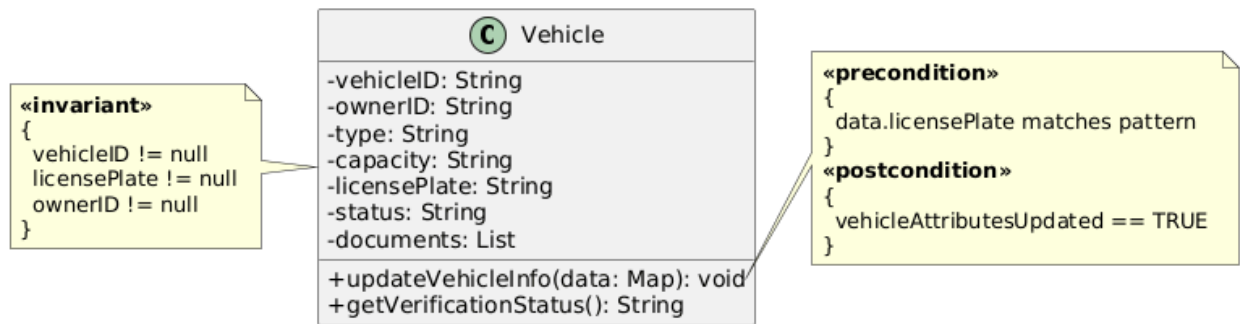


Figure 3.12: Vehicle Entity

3.5. Packages

This section describes the logical grouping of the Kuli subsystems into five domain-driven packages. This organization ensures that the system is modular, allowing for easier maintenance and the ability to scale specific domains independently as the platform grows.

Package Overview

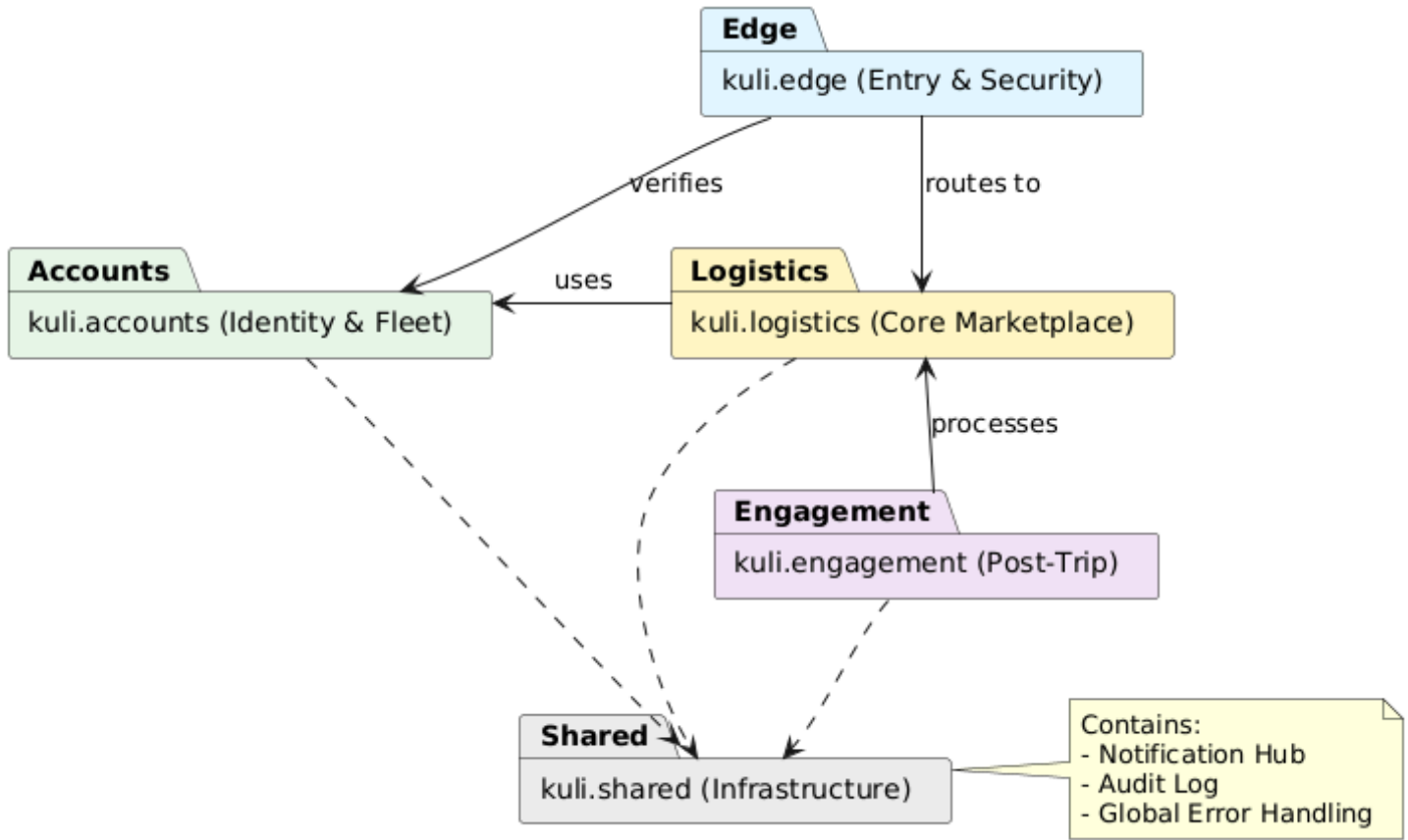
The following table maps the identified subsystems to their respective high-level packages.

Table 3.2: subsystem

Domain Package	Contained Subsystems	Primary Responsibility
kuli.edge	Access & Gateway (AG), Identity (ID)	Entry point security, routing, and user authentication.

kuli.accounts	User Accounts (UA), Truck Registry (TR)	Management of client profiles, truck owner data, and fleet verification.
kuli.logistics	Booking & Pricing (BP), Live Operations (LO)	Core business logic: request matching, pricing, and real-time trip execution.
kuli.engagement	Financial Settlement (PM), Feedback & Disputes (FD)	Post-trip processing: billing, payments, and quality assurance/reviews.
kuli.shared	System Utilities (SU), Common	Shared infrastructure: notifications, logging, and global utilities.

3.5.1 Package Diagram



4 IMPLEMENTATION

This chapter describes the implementation of the KULI logistics platform as delivered in the current codebase. The system is implemented as a modular monorepo containing a Node.js backend API, a React Native mobile application, and a React/Vite administrative dashboard. MongoDB is used as the persistent database through repository classes built on the native MongoDB driver, while Supabase is used for authentication token verification and client-side authentication integration.

The implementation focuses on the core logistics workflow: user registration and role-based access, vehicle registration and verification, quotation, proximity-based matching, KULI request creation, offer handling, manual trip status updates, in-app notifications, cash payment records, ratings, reports, hotline support, and administrative oversight.

4.5.1 Mapping Models to Code

The analysis and design models are mapped into JavaScript modules, service classes, repository classes, shared enumerations, API routes, and frontend screens. Domain concepts such as User, Vehicle, Vehicle Class, KULI Request, Trip Offer, Payment, Rating, Report, Notification, Message, and Hotline Ticket are persisted as MongoDB documents. Behavior is concentrated in service classes, while persistence is isolated inside MongoDB repositories.

Mapping Associations

Associations are implemented using document reference identifiers rather than relational foreign keys. Typical reference fields include `clientId`, `ownerId`, `vehicleId`, `requestId`, `selectedOwnerId`, `selectedVehicleId`, `acceptedOfferId`, `hotlineTicketId`, and `reportedUserId`. Service methods resolve these references when related data is required.

Unidirectional One-to-One Associations

A completed KULI request has one payment record. The relationship is unidirectional because the payment stores requestId, while the KULI request does not store a paymentId. The repository enforces one payment per request using a unique index.

```
await this.collection.createIndexes([
  { key: { requestId: 1 }, unique: true, name: 'payments_request_unique' }
]);
```

The same pattern is used when MarketplaceService or EngagementService creates a cash, pay-on-delivery payment record for a completed request.

```
return this.paymentRepository.save({
  id: createId('pay'),
  requestId: request.id,
  payerClientId: request.clientId,
  payeeOwnerId: request.selectedOwnerId,
  status: paymentStatuses.pending,
  flow: paymentFlows.payOnDelivery,
  method: paymentMethods.cash
});
```

Bidirectional One-to-One Associations

No true bidirectional one-to-one relationship is implemented. The codebase follows a document-oriented design where each document stores only the reference required for the workflow. This avoids duplicate state that would need to be synchronized across collections.

One-to-Many Associations

The implementation contains several one-to-many relationships. A truck owner owns many vehicles, represented by ownerId in the vehicles collection. A client creates many KULI

requests, represented by `clientId` in the `kuli_requests` collection. A request owns many offers, messages, and status events through `requestId`.

Relationship	Implementation
User -> Vehicles	<code>vehicles.ownerId</code> links each vehicle to one truck owner.
Client -> KULI Requests	<code>kuli_requests.clientId</code> links each request to one client.
Request -> Trip Offers	<code>trip_offers.requestId</code> links multiple offers to one request.
Request -> Messages	<code>messages.requestId</code> stores request-bound conversation history.
Request -> Status Events	<code>kuli_status_events.requestId</code> stores trip status history.

```
async listByRequestId(requestId) {
  const documents = await this.collection
    .find({ requestId }, { sort: { createdAt: 1 } })
    .toArray();
  return documents.map(normalize);
}
```

Many-to-Many Associations

The request-to-vehicle matching relationship is implemented as a many-to-many association through the `trip_offers` collection. A request may be sent to several vehicles, and a vehicle may receive many offers over time. Each offer document records the `requestId`, `vehicleId`, `ownerId`, offer status, distance at offer time, ETA, and expiry time

```
await this.collection.createIndexes([
  { key: { requestId: 1, vehicleId: 1 }, unique: true,
    name: 'trip_offers_request_vehicle_unique' }
]);
```

```
const offers = await this.tripOfferRepository.insertMany(
  eligibleVehicles.map((vehicle) => ({
    id: createId('offer'),
    requestId: request.id,
    ownerId: vehicle.ownerId,
    vehicleId: vehicle.id,
    status: offerStatuses.sent,
    expiresAt
  )))
);
```

Mapping Operation Contracts to Exceptions

Operation contracts are implemented through route guards, role checks, account status checks, input validation, and service-level business rules. Controlled failures use the `AppError` class, which carries an HTTP status code, a machine-readable code, a message, and optional details

.

```
export class AppError extends Error {
  constructor(statusCode, code, message, details = undefined) {
    super(message);
    this.statusCode = statusCode;
    this.code = code;
    this.details = details;
  }
}
```

Authentication is resolved from the Authorization header using the Supabase token verifier, then mapped to a local MongoDB user profile. Role authorization is enforced by `assertRole`, and blocked accounts are rejected by the account-status guard.

```

const authUser = await context.tokenVerifier
  .verifyAuthorizationHeader(authorization);
const currentUser = await context.userRepository
  .findBySupabaseUserId(authUser.sub);

if (!allowedRoles.includes(user.role)) {
  throw new AppError(403, 'FORBIDDEN',
    'You do not have access to this resource.');
```

Validation rules are placed close to the business operation. For example, quote creation requires valid GeoJSON pickup and destination points, valid address text, an active vehicle class, an active pricing rule, and load values that do not exceed the selected vehicle class capacity.

```

if (!location?.point || location.point.type !== 'Point' ||
  !Array.isArray(location.point.coordinates)) {
  throw new AppError(400, 'LOCATION_REQUIRED',
    `${fieldName} requires GeoJSON coordinates.`);
}
```

Business rule violations are rejected with conflict or validation errors. Offer acceptance checks offer ownership, status, expiry, vehicle availability, and request availability before committing the accepted request.

```

if (![offerStatuses.sent, offerStatuses.viewed].includes(offer.status)
  || offer.expiresAt <= now) {
  throw new AppError(409, 'OFFER_NOT_AVAILABLE',
    'This offer is no longer available.');
```

```
const busyVehicle = await this.vehicleRepository.markBusyIfAvailable({
  vehicleId: offer.vehicleId,
  activeTripId: offer.requestId
});
```

Trip progress is controlled by a fixed transition map. This ensures that users cannot jump from an invalid state to another invalid state during manual trip tracking.

```
const statusTransitions = {
  pending: ['cancelled', 'timed_out'],
  accepted: ['en_route_to_pickup', 'cancelled'],
  en_route_to_pickup: ['arrived_at_pickup', 'cancelled'],
  loading: ['in_transit', 'cancelled'],
  in_transit: ['unloading', 'cancelled'],
  unloading: ['completed', 'cancelled']
};
```

Mapping the Class Model to the Storage Schema

The class model is mapped to MongoDB collections through repository classes. Each repository encapsulates one collection and exposes methods such as save, findById, list, listByOwnerId, findApprovedAvailableNearby, transitionStatus, and insertMany. MongoDB `_id` values are normalized into the application id field before data is returned to services.

```
const normalize = (document) => {
  if (!document) return null;
  const { _id, ...rest } = document;
  return { id: String(_id), ...rest };
};
```

Collection	Main purpose
users	Profiles, roles, account status, Supabase ID, contact data, preferences, owner metadata.
vehicles	Owner vehicles, class references, plate, verification status, availability, location.
vehicle_classes	Configurable truck categories and default pricing capacity data.
vehicle_documents	Identity, license, registration, ownership, and insurance document links.
files	Stored file metadata and entity links for vehicle, report, profile, and message files.
pricing_rules	Versioned fare rules, vehicle-class pricing, fuel surcharge, and load adjustments.
kuli_requests	Shipment requests, locations, load data, selected owner/vehicle, accepted offer, status, quote snapshot.
trip_offers	Association documents between requests and candidate vehicles.
kuli_status_events	Status transition audit trail for trips.
messages	Request-scoped communication history.
payments	Cash/manual payment records linked to completed requests.
ratings	Client/assistant ratings and reviews for truck owners.
reports	Complaints, disputes, evidence links, and admin resolution data.
notifications	In-app notification records.
device_tokens / notification_intents	Device token and external notification intent records.

hotline_tickets	Call-assisted booking and support ticket workflow.
audit_logs	Administrative and sensitive operation audit history.

The implementation uses references for entities that change independently, such as users, vehicles, requests, offers, payments, and reports. It uses embedded snapshots where historical consistency matters, such as `vehicleClassSnapshot` inside a vehicle and `quoteSnapshot` inside a KULI request. This prevents later pricing or vehicle-class changes from altering the historical meaning of an existing trip.

```
await this.collection.createIndexes([
  { key: { 'currentLocation.point': '2dsphere' },
    name: 'vehicles_current_location_2dsphere', sparse: true },
  { key: { licensePlate: 1 }, unique: true,
    name: 'vehicles_license_plate_unique' }
]);
```

4.5.2 Source Codes of Major Classes, Packages, or Interfaces

The codebase is organized around a small set of major components. The backend API performs routing and security checks, service classes implement business workflows, repository classes isolate MongoDB access, and the frontend applications consume the REST API through shared client utilities.

Authentication and Account Management Component

This component connects Supabase authentication with the local KULI user profile. Supabase identifies the authenticated user, while MongoDB stores the application role, account status, contact fields, and preferences.

```
await this.collection.createIndexes([
  { key: { supabaseUserId: 1 }, unique: true },
  { key: { email: 1 }, unique: true, sparse: true },
```

```
    { key: { phone: 1 }, unique: true, sparse: true }  
  ]);
```

API Routing and Middleware Layer

The API layer parses the HTTP method and path, resolves authentication, applies account and role guards, parses the request body, and delegates the operation to the appropriate service.

```
if (method === 'POST' && path === '/api/v1/kuli-requests') {  
  const { currentUser } = await resolveAuth();  
  assertActiveAccount(currentUser);  
  assertRole(currentUser, [roles.client]);  
  return success(await context.marketplaceService.createRequest({  
    actor: currentUser,  
    input: await parseJsonBody(request),  
    idempotencyKey: request.headers['idempotency-key']  
  }), 201);  
}
```

Vehicle Registry Component

The vehicle registry manages vehicle classes, vehicle creation, document upload metadata, owner availability, admin verification, and admin vehicle status decisions.

```
return this.vehicleRepository.save({  
  id: createId('veh'),  
  ownerId: actor.id,  
  vehicleClassId: vehicleClass.id,  
  licensePlate,  
  verificationStatus: verificationStatuses.pending,  
  availabilityStatus: vehicleAvailabilityStatuses.offline,  
  documentsRequired: requiredDocumentTypes  
});
```

Quote and Pricing Component

The quote service validates route locations, calculates distance and ETA through the maps provider, applies the active pricing rule, checks vehicle-class capacity, and ranks candidate vehicles by distance, rating, completion rate, and dispute penalty.

```
const rankingScore = Number((
  distanceKm * 10 - rating * 3 - completionRate * 2 + disputePenalty
).toFixed(2));
```

Marketplace, Request, and Offer Component

MarketplaceService implements the main booking workflow. It creates requests, sends offers to selected eligible vehicles, records status events, manages messages, accepts offers, cancels requests, and creates payment records when trips complete.

```
const request = await this.kuliRequestRepository.save({
  id: createId('kreq'),
  requestCode: requestCode(),
  clientId: actor.id,
  status: kuliStatuses.pending,
  pickupLocation: input.pickupLocation,
  destinationLocation: input.destinationLocation,
  quoteSnapshot: quote.quoteSnapshot
});
```

Payment, Rating, and Report Component

EngagementService handles cash payment confirmation, payment disputes, ratings, report submission, evidence upload metadata, admin report resolution, and rating recalculation.

```
if (!Number.isInteger(input.rating) || input.rating < 1 || input.rating > 5) {
  throw new AppError(422, 'INVALID_RATING',
    'Rating must be an integer between 1 and 5.');
```

```
}
```

Notification Component

Notifications are implemented as in-app records stored in MongoDB and are delivered through the platform notification interface.

```
const createNotification = ({ recipientUserId, type, title, body, data }) => ({
  id: createId('notif'),
  recipientUserId,
  type,
  title,
  body,
  data,
  channels: ['in_app'],
  deliveryStatus: 'pending'
});
```

Support and Hotline Component

The support component manages hotline tickets for call-assisted booking. Tickets track caller details, source, status, assigned assistant, request link, and ticket history.

```
await this.collection.createIndexes([
  { key: { ticketCode: 1 }, unique: true },
  { key: { assignedAssistantId: 1, status: 1 } },
  { key: { callerPhone: 1, createdAt: -1 } }
]);
```

Admin Dashboard Component

The admin dashboard is implemented with React/Vite. It communicates with the API through the KULI API client and supports operational views such as users, vehicles, verification, pricing, requests, reports, payments, and system health.

```
const usersQuery = useQuery({
  queryKey: ['admin-users'],
  queryFn: async () => {
    const result = await kuliApi.request('/admin/users');
    return result.data;
  },
  enabled
});
```

Mobile Application Component

The mobile application is implemented with React Native. It contains role-specific workflows for clients and truck owners, including quote creation, request management, owner offers, active trip updates, payments, ratings, reports, notifications, and reusable UI components.

```
export function PrimaryButton({ title, onPress, disabled }: PrimaryButtonProps) {
  return (
    <Pressable onPress={onPress} disabled={disabled}>
      <Text>{title}</Text>
    </Pressable>
  );
}
```

4.5.3 Screen Images

The following screenshots should be inserted from the running mobile application and admin dashboard.

KULI

Move with verified trucks.

Book, verify, accept, and track logistics work with your KULI account.

Local demo mode

Login

Register

SECURE SIGN IN

Use your KULI account.

Sign in to continue to your KULI workspace.

Email

you@example.com

Password

Minimum 6 characters

Forgot password?

Login

Figure 4.5.3-1: Mobile login page

KULLI: P2P Logistics Mobile Application On Demand Trucking Service

KULLI REQUEST

Plan your move.

Static route preview with Addis Ababa area selection.

OpenStreetMap preview / Pickup: Atlas

1Route

2Truck

3Quote

4Sent

STEP 2

Truck and load.

Pick a truck class, describe the load, then get your quote.

HomeRequestActivityNotifications

Small Pickup

Light pickup suitable for appliances, boxes, and small furniture.

700kgCAPACITY

5m3VOLUME

Medium Truck

Medium truck suitable for household relocation and bulky goods.

2000kgCAPACITY

15m3VOLUME

Large Truck

Large truck suitable for commercial and industrial goods.

5000kgCAPACITY

30m3VOLUME

Household

Full move with mixed items.

Furniture

Sofas, beds, tables, cabinets.

Appliance

Fridge, washer, oven, electronics.

Business

Shop stock or repeat delivery.

Weight kg

150

Volume m3

3

Pickup time

Tue, Jun 2

9:00 AM

Selected pickup: Tue, Jun 2 at 9:00 AM

Loading help

Add loading or unloading help to this request.

Yes

Handling notes

fragile

Tip RTB

30

Back

Get quote

HomeRequestActivityNotifications

Figure 4.5.3-2: Client quote creation screen

Time	ETB 0.00
Load adjustment	ETB 150.00
Fuel surcharge	ETB 43.87
Tip	ETB 30.00

Payment is handled after delivery in the current KULI flow. Keep final payment coordination inside the trip chat.

Nearby verified trucks

Select one or more. The first truck to accept gets the trip.



AA-123456

Small Pickup

★★★★★

Selected

0.0
RATING

44.9
MATCH

700kg
CAPACITY

1 selected. KULI will send the request to those trucks.

Edit

Send request


Home

 Request


Activity


Notifications

Figure 4.5.3-3: Candidate vehicle selection

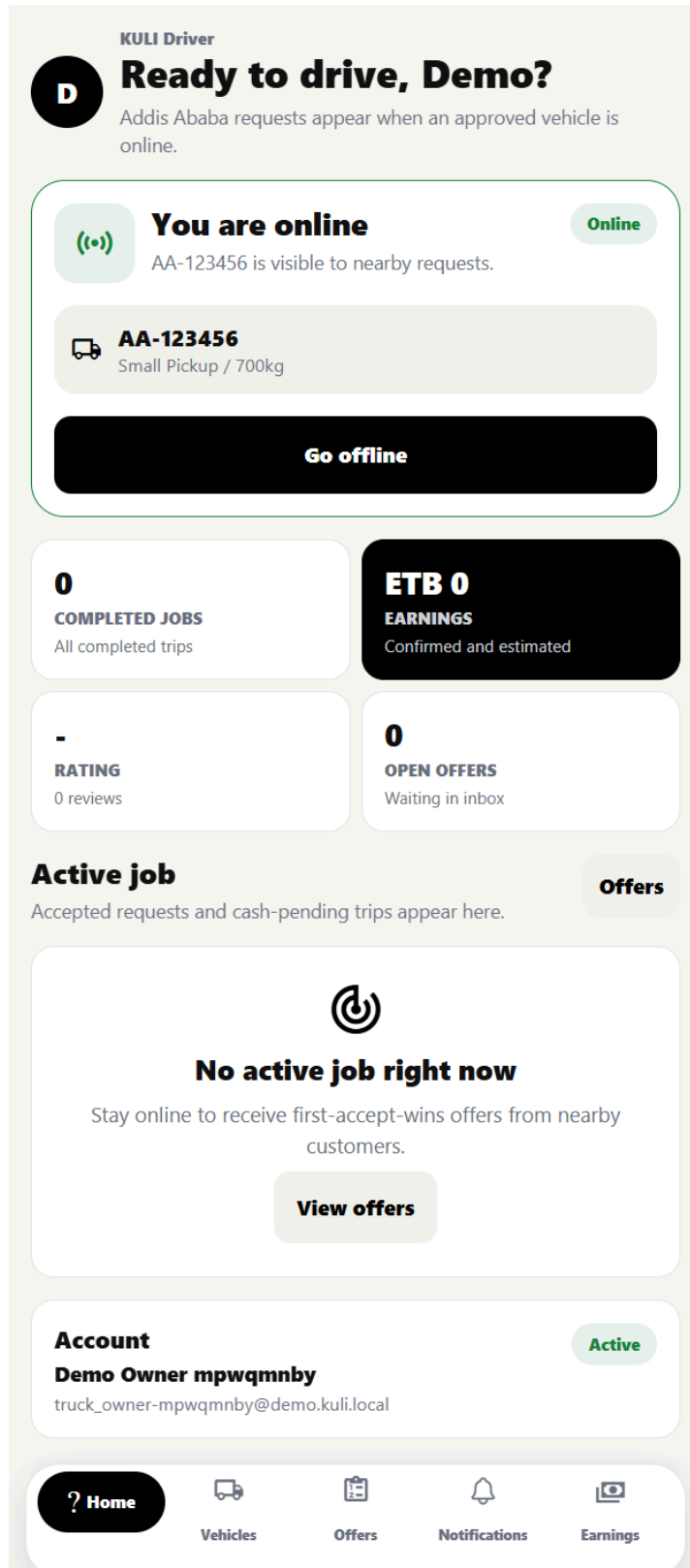


Figure 4.5.3-4: Truck owner dashboard

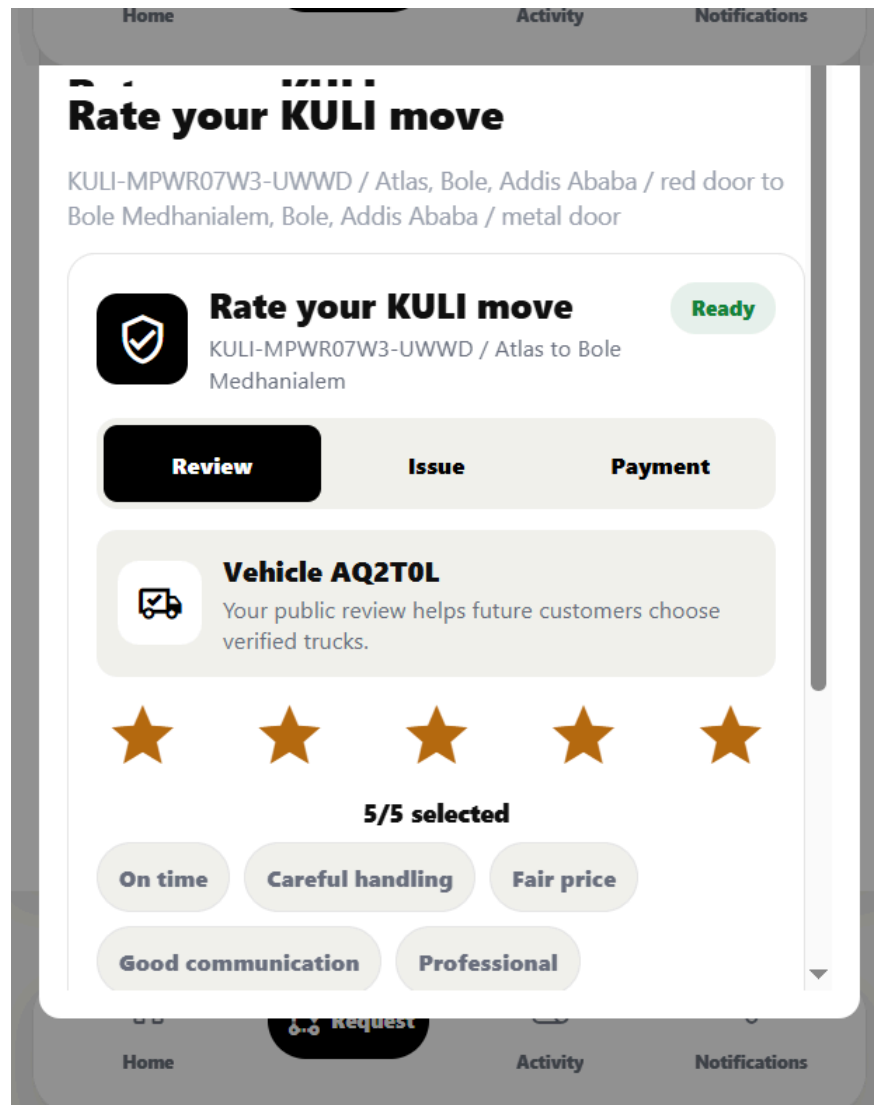


Figure 4.5.3-6: rating/report screen.

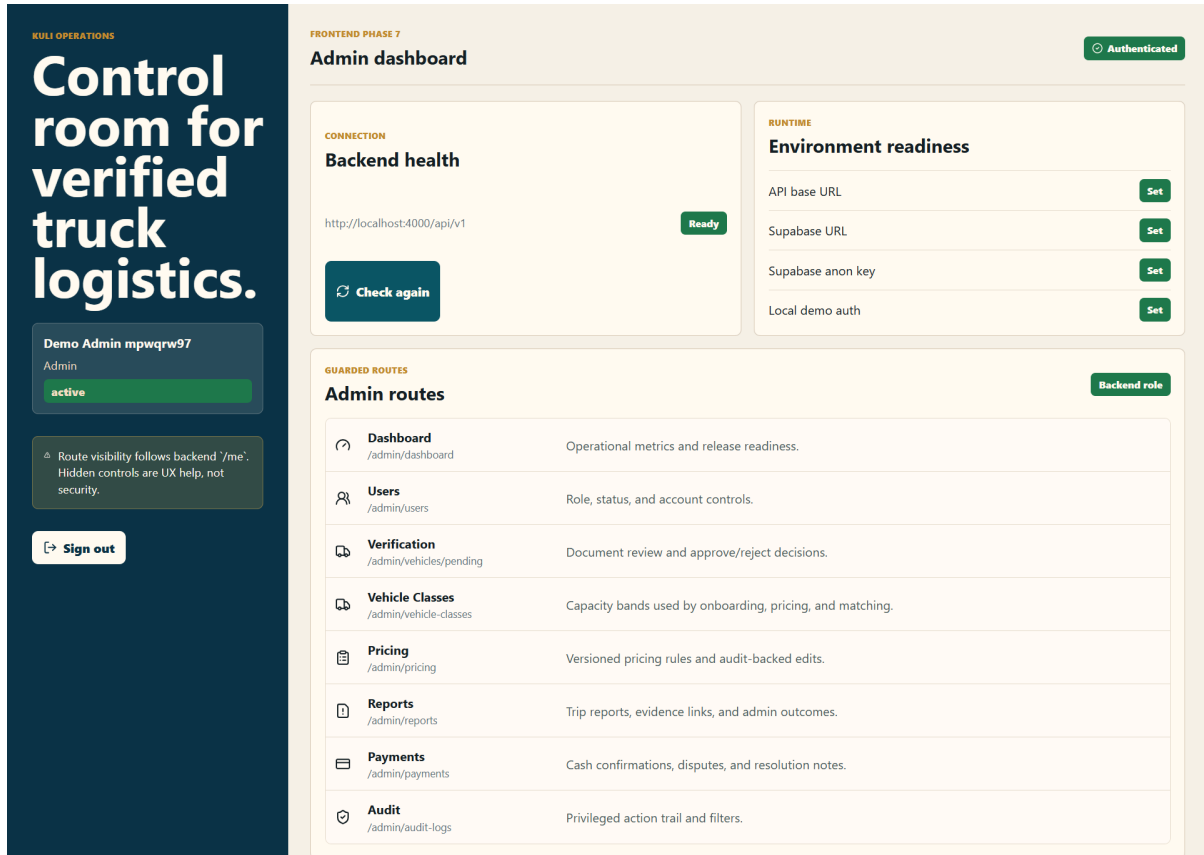


Figure 4.5.3-7: Admin dashboard overview screen

VERIFICATION QUEUE

Pending vehicles

1 pending

AA-134445

Small Pickup

pending

AA-134445

Small Pickup / 700kg / 5m3

No owner notes submitted.

No documents attached yet.

Decision reason

Required for rejection; useful for approval notes.

Approve

Reject

pending

VEHICLE CLASSES

Capacity bands

3 active

New vehicle class

Create a capacity band for owners and quote rules.

New class

Vehicle classes feed owner onboarding, load validation, matching, and pricing rules.

Draft

Name	Slug	Display order
------	------	---------------

Figure 4.5.3-8: Admin vehicle verification screen.

104

TRUST AND FINANCE

Reports and payment disputes

0 reports / 2 payments

Report status

Category

open

All

No reports match this filter.

Select a report

Evidence and outcomes appear after selecting a report.

ETB 951.22

kreq_0ae6guih / owner usr_demo_yc1ab1dg

pending

ETB 1075.25

kreq_enqyj33r / owner usr_demo_uzrvbo5

confirmed by owner

ETB 1075.25

Cash confirmations and disputes stay manual in v1.

Request kreq_enqyj33r / client usr_demo_7wtmx9tp / method cash

confirmed by owner

Confirmed amount

1075.25

Resolution note

Required audit note for the payment decision.

Resolve payment

Figure 4.5.3-9: Admin pricing or reports management screen.

Choose or create a ticket to begin assisted booking.

All

+251...

No hotline tickets match the filter.

Create ticket

Incoming

Missed

Manual

Caller phone

+251911111111

Follow up

2026-05-21T12:00:00.000Z

Create ticket

Call notes

Caller context, building access, load notes, confirmation details.

Client lookup

Search existing clients by phone; assisted bookings can still use a caller snapshot when no app profile exists.

Search

Phone

+251911111111

No linked client selected. The request will keep caller contact snapshot.

Assisted quote

Use manual coordinates while maps stay provider-backed on the API.

Quote

Pickup

Bole, Addis Ababa

Pickup lon

38.7903

Pickup lat

8.9806

Destination

Piassa, Addis Ababa

Destination lon

38.7578

Destination lat

9.0350

Pickup time

Today, flexible

Vehicle class

Small Pickup

Load type

household move

Weight kg

800

Volume m3

8

Figure 4.5.3-10: Hotline/support ticket management screen.

5. TEST PLAN DOCUMENT

5.1 Introduction

In this project, testing was conducted to verify that the “**The KULLI Platform**” functions correctly according to the identified functional and non-functional requirements.

The testing process focused on validating major system components such as authentication, vehicle registration and verification, quote generation and dynamic pricing, geospatial vehicle matching, status tracking, payment settlement, call-center assisted booking, notifications, and client–driver support interactions.

5.2 Purpose

The purpose of testing KULLI is to verify reliability, correctness, usability, security, and workflow and completeness. The test plan specifically checks that:

- Clients can register, sign in, request trucks, receive quotes, send requests, track status, message, cancel, rate, report, and view notifications.
- Truck owners can register, create vehicles, upload documents, manage availability, receive offers, accept or decline, update trip status, confirm payment, and view earnings/ratings.
- Staff-only workflows remain in the web dashboard and are not exposed as mobile registration options.

5.3. Features to be tested/not to be tested

Feature	Description	Risk
Authentication and registration	Login, public client/truck-owner registration, logout, email confirmation handling, session persistence.	High
Password reset/account recovery	Forgot password email flow through Supabase and invalid email handling.	Medium
Role-based routing	Clients route to client mobile tabs; truck owners route to owner mobile tabs; admin/assistant are blocked on mobile.	High
Client request creation	Pickup/drop-off, load details, truck type, schedule, quote, candidate selection, and dispatch.	High
Quote calculation	Distance, ETA, pricing rule snapshot, ETB total, and price breakdown.	High
Nearby truck/candidate discovery	Approved, online vehicles are returned and unverified/offline vehicles are excluded.	High
Request dispatch	KULI request and offer records are created only after backend confirmation.	High
Owner vehicle registration	Vehicle class, license plate, capacity, notes, and duplicate/missing-field handling.	High
Document upload/verification state	Required document capture, upload intent, metadata, pending/rejected/approved states.	High

KULLI: P2P Logistics Mobile Application On Demand Trucking Service

Owner availability	Approved vehicles can go online; unapproved vehicles cannot.	High
Offer inbox	Owner receives offers with route/load/estimate and can view details.	High
Accept/decline offer	First-accept-wins, conflict handling, and decline behavior.	High
Active trip status updates	Owner can move only through valid manual status transitions.	High
Client tracking/status timeline	Client sees status history and active trip details without fake live GPS.	High
Messaging	Request-scoped messages send, refetch, and archive after terminal states.	Medium
Cancellation	Client can cancel allowed requests with reason and cannot cancel closed states.	High
Rating/review	Eligible completed/cancelled trips can be rated with clear UI feedback.	Medium
Report/issue/dispute	Client can file report, attach optional evidence, and dispute eligible payment.	High
Manual cash payment confirmation	Owner confirms cash after completion; pending/disputed states display clearly.	High

Notifications and preferences	In-app notification list, unread state, mark read, and preference update.	Medium
Loading/empty/error/offline states	UI handles slow API, no data, validation errors, network-style failures, and retry paths.	Medium
Mobile UI/UX responsiveness	Bottom tabs, map preview, request flow, cards, forms, and text fit on small screens and Expo web.	Medium

Table 5.1 tested feature

5.2.2 Features Not to Be Tested in This Version

Feature	Reason
Live GPS tracking	Current v1 uses static map previews and manual status updates.
Real-time moving truck animation	Not implemented and must not be implied by the UI.
Real digital payment gateway	v1 records cash/manual payment confirmation only.
Pay-in-advance, Pay-on-acceptance	Not supported by current backend workflow.

Real SMS/email/push provider delivery	External provider delivery is not guaranteed unless provider keys are configured. In-app notification intent can be tested.
Advanced toll/live fuel pricing	Current pricing uses configured pricing rules and static fuel surcharge logic where implemented.
Automated document OCR/verification	Admin manual review and file metadata are in scope; OCR is future scope.
Multi-city marketplace expansion	Current mobile flow is designed around Addis Ababa locations.

Table 5.2 not tested features

4.3. Pass/ Fail criteria

A test case is marked as Pass when the actual result matches the expected result and the application behaves without unexpected errors. A test case is marked as Fail when the actual result differs from the expected result, when an error is not recoverable, or when a required validation or workflow rule is not enforced. The P column indicates passed test cases, and the F column indicates failed test cases.

Criterion	Pass	Fail
Expected output	Actual output matches the expected result in the test case.	Actual output differs, is incomplete, or cannot be observed.

Backend confirmation	State-changing actions pass only when the backend confirms success and the UI reflects returned state.	UI shows success without confirmed backend state or ignores server rejection.
No fake success states	Request creation, offer acceptance, payment confirmation, rating, report, and status update show success only after API success.	UI pretends an action succeeded before the server confirms it.
Error visibility	Validation, permission, conflict, and network errors are visible and recoverable.	Error is hidden, unclear, or forces the user to restart unnecessarily.
Security/role enforcement	Mobile hides privileged staff registration and backend blocks unauthorized commands.	Unauthorized user can access or perform restricted actions.
Data persistence	Created records remain available after refresh/re-login where expected.	Data disappears, duplicates unexpectedly, or is tied only to local UI state.
Usability	UI labels are clear, touch targets usable, and screen content fits on supported web/mobile viewports.	UI contains route/debug labels, clipped text, confusing status, or inaccessible actions.

Table 5.5pass/fail criteria

4.4. Approach / Strategy

The approach is designed as an acceptance-level and project-level strategy because KULI includes a mobile app, backend API, database, Supabase authentication, and an admin web dashboard. Testing therefore combines automated checks, integration tests, and role-based manual workflow testing.

The selected strategy is risk-based. Features that can change money, request status, vehicle availability, user roles, or trust records are tested more heavily than display-only areas. The main rule is that the UI must never show success for important actions until the backend confirms the action.

4.4.1 Overall Testing Approach

The testing strategy follows a layered verification approach that aligns with the architecture of the KULI system. Different testing levels are applied to ensure that individual components function correctly, that integrated modules communicate properly, and that complete user workflows satisfy the specified requirements.

Unit / Business Rule Layer: Automated tests are used to verify business logic, including validation rules, pricing calculations, role-based access control, and other core application rules.

Integration / API Layer: Integration testing is performed to validate communication between system components, including profile synchronization, vehicle verification processes, quote generation, request management, and authentication services.

Mobile UI / User Layer: Manual testing is conducted to verify user interactions within the mobile application, including client request workflows, vehicle management, notifications, trip tracking, and earnings-related features.

Regression Layer: Regression testing is performed after significant code modifications to ensure that existing functionality continues to operate correctly and that new changes do not introduce unintended defects.

Acceptance Layer: End-to-end testing is conducted to validate complete workflows involving different user roles. This includes scenarios such as request creation, offer acceptance, trip completion, payment confirmation, and user feedback processes.

4.4.2 Tools

The testing process utilizes commonly available development and testing tools. This includes:

Node.js Test Runner (npm test) and TypeScript Compiler: These tools are used to execute automated backend tests and verify TypeScript correctness throughout the application.

Repository Scripts (npm run lint and npm run typecheck): These scripts validate coding standards, verify project structure, and ensure consistency across shared contracts and application modules.

Expo CLI, Android Emulator, and Expo Go: These tools are used to build, deploy, and test the mobile application on web browsers, Android emulators, and physical devices.

Docker Compose and Supabase Dashboard: Docker Compose is used to manage local infrastructure services such as MongoDB and Redis, while the Supabase Dashboard supports authentication testing, user management, email verification, and password reset validation.

Browser Developer Tools: Standard browser development tools are used for debugging, inspecting application behavior, monitoring network requests, and identifying user interface issues.

4.4.3 Metrics Collected

The following metrics are collected throughout the testing process to evaluate system quality and testing progress:

Test Case Pass Rate (per feature and overall): Calculated as the number of passed test cases divided by the total executed test cases. This metric is collected at all testing levels.

Defect Count by Severity: Defects are classified as Critical, High, Medium, or Low and tracked throughout the testing process to support prioritization and resolution.

Defect Discovery Rate: The number of new defects identified during each testing cycle. This metric helps assess the quality trend of the system.

Defect Resolution Rate: The number of defects resolved compared to the number of newly discovered defects during a testing cycle.

Requirements Coverage: Measures the percentage of functional and non-functional requirements that have at least one corresponding test case. High-priority requirements should achieve 100% test coverage.

Automated Test Coverage: Tracks the number of automated unit and integration tests executed for backend services and business logic components.

Critical Workflow Success Rate: Measures the successful completion of key workflows such as authentication, vehicle registration, request creation, offer acceptance, trip completion, payment confirmation, and rating submission.

Blocked Test Count: Records test cases that cannot be executed because of environment, dependency, configuration, or external service issues.

Regression Test Pass Rate: Measures the percentage of regression test cases that pass after system modifications, helping identify whether changes introduce new defects.

MTBF (Mean Time Between Failures) and formal Software Reliability Engineering (SRE) metrics were not applied because the project is not operating in a long-term production environment. Instead, system reliability was evaluated through automated test pass rates, successful completion of critical workflows, startup readiness checks, error monitoring, and regression testing.

4.4.4 Configuration Management

Configuration management is maintained through Git version control, environment configuration files, and documented testing environments. All significant development changes are committed using descriptive commit messages, and test results are associated with the corresponding branch name or commit hash to ensure traceability.

Environment-specific settings are managed using **.env** files, while example configurations are provided through **.env.example** files where applicable. Sensitive information, such as Supabase service keys and other credentials, is excluded from test documentation and source control. Docker Compose is used to provide a consistent and repeatable local testing environment, while demo authentication is restricted to development and UI testing purposes. Final validation is performed using the standard Supabase authentication process.

When defects are identified, testers record the build or commit tested, user role, testing environment, reproduction steps, expected result, actual result, defect severity, and any relevant screenshots or logs. This information supports effective defect tracking, analysis, and resolution throughout the testing process.

4.4.5 Configurations Tested

The KULI system is tested in multiple configurations to ensure correct operation across its mobile application, backend services, database, authentication platform, and administrative dashboard. Testing is performed using a local backend environment running through Docker Compose with MongoDB and Redis, which serves as the primary configuration for backend and API validation. The mobile application is tested through Expo Web using a desktop browser and through Android emulators or physical devices using Expo Go to verify responsiveness and user interactions. Authentication functionality is tested using Supabase to validate user registration, login, email verification, password reset, and session management features. A local demo authentication configuration is also used during development to support repeatable testing without external authentication limitations. In addition, the browser-based admin dashboard is tested to verify administrative functions, user management, support operations, and system monitoring features.

The primary hardware used for testing consists of a developer laptop or desktop computer. The software environment includes Node.js, React Native, Expo, MongoDB, Redis, Supabase, and browser-based administrative tools. Important configuration combinations tested include the mobile application connected to the local API and MongoDB/Redis services, the mobile application integrated with Supabase authentication, the admin dashboard connected to the local backend services, and interactions between client and owner roles operating through the same backend environment.

4.4.6 Regression Testing Levels

Regression testing is performed after significant frontend or backend modifications to ensure that existing functionality continues to operate correctly following system changes. Multiple levels of regression testing are applied based on the nature and impact of the modification.

Critical Backend Regression: Automated backend tests are executed using the project test suite to verify the stability and correctness of server-side functionality.

Mobile Build Regression: TypeScript validation and Expo verification checks are performed to ensure that the mobile application builds and runs successfully after code changes.

API Contract Regression: Linting, type-checking, and startup validation are performed to verify API consistency and maintain compatibility between system components.

User Interface Regression: Major application screens are manually reviewed to identify issues related to layout, navigation, status updates, and user interaction.

End-to-End Workflow Regression: Complete workflows involving client, owner, and administrative roles are re-executed after major system changes to verify that integrated features continue to function as expected.

The extent of regression testing is determined by defect severity. Critical defects require full regression testing of the affected workflow and related backend functionality, while high-severity defects require testing of the modified feature and its dependent components.

Medium-severity defects require focused regression around the affected screen, module, or API, whereas low-severity defects such as text or layout corrections require visual verification only.

Priority is given to critical business workflows, including authentication and role-based access control, vehicle verification and availability management, quote and request creation, offer acceptance processes, trip status management, payment confirmation, and rating, reporting, or dispute handling features.

4.4.7 Untestable or Unclear Requirements

Some requirements may be unclear, unavailable, or not testable in the current version. These are handled as follows:

- If a requirement is unclear, it is documented as an open question and clarified before implementation or final acceptance.
- If a requirement conflicts with security, the safer interpretation is used. For example, admin and assistant self-registration is not allowed on mobile.
- If a requirement needs unavailable external services, it is marked as out of scope or partially tested. For example, external SMS/email delivery is not fully tested unless provider keys are configured.
- If a requirement is future scope, the UI must not imply that it exists. For example, KULI v1 uses status-based tracking and static map previews, not live GPS tracking.
- If a requirement cannot be automated, it is added to manual test cases with clear steps and expected results.

4.4.8 Coverage Requirements

As a project-level test plan, testing is performed on the KULI system as an integrated product rather than as a collection of isolated components. Test coverage includes all major functional areas and their interactions to ensure that end-to-end workflows operate correctly across different user roles and system modules.

The covered components include authentication, profile synchronization, and role-based access control; vehicle registration, document upload, verification, and availability management; quote generation, candidate selection, and request dispatching; owner offer management, acceptance workflows, active trips, and status updates; client tracking, messaging, cancellation, and notification services; trip history, ratings, reports, payment dispute handling, and owner earnings management; as well as administrative verification, support, reporting, payment oversight, and operational functions.

Acceptance testing focuses on validating complete business workflows across these components, while individual modules are verified through automated backend tests and targeted user interface testing. The objective is to ensure that all critical system requirements and user interactions are adequately covered by the testing process.

4.4.9 Special Testing Requirements

Several project-specific requirements must be verified during testing to ensure system correctness, security, and consistency. All state-changing actions, such as request creation, offer acceptance, trip status updates, and payment confirmation, must be validated by the backend before success messages are displayed to users. Administrative and assistant functions must remain restricted and must not appear as public registration options within the mobile application. Vehicle availability controls must ensure that unapproved vehicles cannot become active or appear as eligible candidates for requests.

Testing must also verify that the first-accept-wins mechanism is enforced when multiple owners receive the same request. Payment-related workflows must accurately represent cash or manual payments and must not imply the use of digital payment processing. Tracking features must clearly indicate that the system uses status-based tracking and static map information rather than real-time GPS tracking. Error messages should be understandable to users while providing sufficient information for troubleshooting and debugging. In addition, all test accounts, sample data, and testing environments must protect user privacy and must not expose real personal documents, credentials, or production secrets.

4.5. Test cases with specifications

Some selected test cases for the functional requirements are as follows:.

Test Scenario Name: Authentication

Description: This scenario verifies user authentication functions including login, registration, password reset, logout, and session persistence.

Table 4.5-1 Test case for Authentication

#	Test case	Input Data	Expected Result	Actual Result	P	F
1	AUTH-001: Verify valid user can sign in.	Email: client.test@kuli.local; Password: Test123!; Role: Client and Truck Owner tested separately	User is authenticated and routed by backend profile role.	User is authenticated and routed by backend profile role.	✓	
2	AUTH-002: Verify invalid login shows clear error.	Email: unknown.user@kuli.local; Password: WrongPass123!	Login fails; user sees recoverable error; no route change.	Login fails; user sees recoverable error; no route change.	✓	
3	AUTH-003: Verify public client registration.	Name: Abebe Bekele; Email: abebe.client@kuli.local; Phone: +251911223344; Password: Test123!	Client account/profile exists and routes to client tabs after confirmation.	Client account/profile exists and routes to client tabs after confirmation.	✓	
4	AUTH-004: Verify public truck-owner registration.	Name: Hiwot Trucking; Email: hiwot.owner@kuli.local; Phone: +251922334455; Password: Owner123!	Truck owner account/profile exists and routes to owner tabs after confirmation.	Truck owner account/profile exists and routes to owner tabs after confirmation.	✓	

5	AUTH-005: Verify reset request for valid email.	Email: abebe.client@kuli.local	Success message appears and Supabase sends reset email/link.	Success message appears and Supabase sends reset email/link.	✓	
6	AUTH-006: Verify invalid/missing email handling.	Email: client@ or blank value	Clear validation error; no crash.	Clear validation error; no crash.	✓	
7	AUTH-007: Verify user can sign out.	Signed-in user: abebe.client@kuli.local	Session clears and app returns to auth screen.	Session clears and app returns to auth screen.	✓	
8	AUTH-008: Verify session survives app refresh.	Signed-in user: abebe.client@kuli.local; Valid session token	User remains signed in and routes by profile.	User remains signed in and routes by profile.	✓	

Test Scenario Name: Role Routing

Description: This scenario verifies that users are routed to the correct workspace according to their backend profile role.

Table 4.5-2 Test case for Role Routing

#	Test case	Input Data	Expected Result	Actual Result	P	F
1	ROLE-001: Verify client routes to client tabs.	Client account: abebe.client@kuli.local	Bottom tabs show Home, Request, Activity, Notifications.	Bottom tabs show Home, Request, Activity, Notifications.	✓	

2	ROLE-002: Verify truck owner routes to owner tabs.	Owner account: hiwot.owner@kuli.local	Bottom tabs show Home, Vehicles, Offers, Notifications, Earnings.	Bottom tabs show Home, Vehicles, Offers, Notifications, Earnings.	✓	
3	ROLE-003: Verify admin blocked from mobile.	Admin account: admin.staff@kuli.local	Mobile shows forbidden/right workspace state; no marketplace tabs.	Mobile shows forbidden/right workspace state; no marketplace tabs.	✓	
4	ROLE-004: Verify assistant blocked from mobile.	Assistant account: assistant.staff@kuli.local	Mobile shows forbidden/right workspace state; no marketplace tabs.	Mobile shows forbidden/right workspace state; no marketplace tabs.	✓	
5	ROLE-005: Verify demo login does not silently change owner to client.	Existing owner email: hiwot.owner@kuli.local	Existing backend role is preserved unless explicit registration changes it.	Existing backend role is preserved unless explicit registration changes it.	✓	

Test Scenario Name: Client Request

Description: This scenario verifies client quote generation, request validation, candidate selection, request dispatch, and map preview behavior.

Table 4. 5-3 Test case for Client Request

#	Test case	Input Data	Expected Result	Actual Result	P	F
1	REQ-001: Verify quote generation.	Pickup: Bole; Destination: Piazza; Truck: Medium Truck; Load: 800 kg; Schedule: Today 2:00 PM	Quote step shows ETB total, route, breakdown, distance/ETA, candidates or empty state.	Quote step shows ETB total, route, breakdown, distance/ETA, candidates or empty state.	✓	

KULLI: P2P Logistics Mobile Application On Demand Trucking Service

2	REQ-002: Verify same pickup/drop-off blocked.	Pickup: Bole; Destination: Bole	UI shows validation error and does not continue.	UI shows validation error and does not continue.	✓	
3	REQ-003: Verify selected truck state.	Truck options: Medium Truck and Large Truck; Selected: Medium Truck	Selected truck card is visibly active and quote payload uses selected class.	Selected truck card is visibly active and quote payload uses selected class.	✓	
4	REQ-004: Verify required load details.	Weight: blank or -10 kg; Volume: blank; Truck: Medium Truck	Backend or UI returns clear recoverable validation error.	Backend or UI returns clear recoverable validation error.	✓	
5	REQ-005: Verify selecting nearby trucks.	Candidate IDs: VEH-101, VEH-102; Selected: VEH-101	Selected state updates and Send request disabled when none selected.	Selected state updates and Send request disabled when none selected.	✓	
6	REQ-006: Verify request creation.	Quote ID: Q-1001; Candidate IDs: VEH-101; Route: Bole to Piazza	Backend creates request/offers; UI shows sent/searching state; Home active move updates.	Backend creates request/offers; UI shows sent/searching state; Home active move updates.	✓	
7	REQ-007: Verify empty discovery handling.	Pickup: Sendafa; Destination: Legetafo; Truck: Large Truck	UI shows no nearby approved trucks and suggests adjustment.	UI shows no nearby approved trucks and suggests adjustment.	✓	
8	REQ-008: Verify recoverable request error.	API endpoint unavailable during quote submission; Pickup: Bole; Destination: CMC	Clear error appears; form data remains available.	Clear error appears; form data remains available.	✓	

9	REQ-009: Verify map-first request UI.	Pickup: Atlas; Destination: Mexico; Map preview enabled	Map preview is visible and central; UI does not claim live GPS.	Map preview is visible and central; UI does not claim live GPS.	✓	
---	--	---	---	---	---	--

Test Scenario Name: Owner Vehicle

Description: This scenario verifies truck owner vehicle registration, document handling, approval status, and availability control.

Table 4.5-4 Test case for Owner Vehicle

#	Test case	Input Data	Expected Result	Actual Result	P	F
1	VEH-001: Verify owner can submit vehicle.	Plate: AA-12345; Class: Medium Truck; Capacity: 3,000 kg; Owner: hiwot.owner@kuli.local	Vehicle appears in My vehicles with pending/draft state.	Vehicle appears in My vehicles with pending/draft state.	✓	
2	VEH-002: Verify missing required documents are visible.	Vehicle: AA-12345; Documents: none uploaded	Required document count shows missing items; submit all unavailable until documents attached.	Required document count shows missing items; submit all unavailable until documents attached.	✓	
3	VEH-003: Verify document upload metadata.	Document type: registration certificate; File: vehicle_registration_ test.pdf	Document shows latest upload metadata and pending review state.	Document shows latest upload metadata and pending review state.	✓	
4	VEH-004: Verify pending state.	Vehicle: AA-12345; Review status: Pending	Vehicle shows Pending and cannot receive offers.	Vehicle shows Pending and cannot receive offers.	✓	
5	VEH-005: Verify	Vehicle: AA-12345; Approval status:	Availability becomes Online	Availability becomes Online	✓	

	approved vehicle can go online.	Approved; Availability: Offline before test	and vehicle can appear in matching.	and vehicle can appear in matching.		
6	VEH-006: Verify gate by approval.	Vehicle: AA-54321; Approval status: Pending	UI/API blocks online state and explains approval required.	UI/API blocks online state and explains approval required.	✓	
7	VEH-007: Verify rejection reason visibility.	Vehicle: AA-54321; Rejection reason: Insurance document unreadable	Rejection reason appears and owner can prepare corrections.	Rejection reason appears and owner can prepare corrections.	✓	

Test Scenario Name: Offer and Trip Execution

Description: This scenario verifies offer reception, acceptance, rejection, concurrency control, and trip status workflow execution.

Table 4. 5-5 Test case for Offer and Trip Execution

#	Test case	Input Data	Expected Result	Actual Result	P	F
1	OFFER-001: Verify owner sees offer.	Offer ID: OFF-2001; Route: Bole to Piazza; Estimate: ETB 2,800	Offer card shows route, load, estimate, distance/expiry.	Offer card shows route, load, estimate, distance/expiry.	✓	
2	OFFER-002: Verify owner acceptance.	Offer ID: OFF-2001; Vehicle: AA-12345; Status: Pending	Request becomes accepted; active trip appears; competing offers close.	Request becomes accepted; active trip appears; competing offers close.	✓	
3	OFFER-003: Verify owner decline.	Offer ID: OFF-2002; Vehicle: AA-12345; Status: Pending	Offer removed/marked declined; request remains open for other offers.	Offer removed/marked declined; request remains open for other offers.	✓	

4	OFFER-004: Verify race-condition behavior.	Owner A: hiwot.owner@kuli.local; Owner B: second.owner@kuli.local; Request: REQ-3001	Owner A wins; Owner B gets conflict/unavailable message.	Owner A wins; Owner B gets conflict/unavailable message.	✓	
5	OFFER-005: Verify valid manual trip flow.	Request: REQ-3001; Status sequence: en route, arrived, loading, in transit, unloading, completed	Each allowed transition succeeds and creates timeline event.	Each allowed transition succeeds and creates timeline event.	✓	
6	OFFER-006: Verify invalid status blocked.	Request: REQ-3001; Invalid transition: Pending directly to Completed	Backend rejects and UI shows clear error.	Backend rejects and UI shows clear error.	✓	
7	OFFER-007: Verify accepted trip shows for both roles.	Accepted request: REQ-3001; Client and owner sessions	Both users see active trip details and current status.	Both users see active trip details and current status.	✓	

Test Scenario Name: Tracking and Messaging

Description: This scenario verifies active trip tracking, timeline updates, request-based messaging, and terminal chat behavior.

Table 4.5-6 Test case for Tracking and Messaging

#	Test case	Input Data	Expected Result	Actual Result	P	F
1	TRACK-001: Verify client sees active tracking workspace.	Accepted request: REQ-3001; Client: abebe.client@kuli.local	Map preview, route, status, timeline, and messages are visible.	Map preview, route, status, timeline, and messages are visible.	✓	
2	TRACK-002: Verify status	Status update: In transit;	Timeline shows new event with timestamp/status.	Timeline shows new event with timestamp/status.	✓	

	events visible.	Timestamp: current test time				
3	TRACK-003: Verify request-scope d chat.	Message: Please call me when you reach Megenagna; Request: REQ-3001	Message appears for both parties after API confirmation.	Message appears for both parties after API confirmation.	✓	
4	TRACK-004: Verify retryable error.	Message: Test retry message; API intentionally unavailable	Error/retry state appears; no fake sent state.	Error/retry state appears; no fake sent state.	✓	
5	TRACK-005: Verify terminal behavior.	Terminal request: REQ-3002; Status: Cancelled	Messaging is collapsed/archive d with follow-up guidance.	Messaging is collapsed/archive d with follow-up guidance.	✓	

Test Scenario Name: Cancellation

Description: This scenario verifies request cancellation rules, reason validation, and cancelled trip history behavior.

Table 4.5-7 Test case for Cancellation

#	Test case	Input Data	Expected Result	Actual Result	P	F
1	CANCEL-001: Verify pending cancellation.	Request: REQ-4001; Status: Pending; Reason: Plans changed	Request becomes Cancelled; open offers close; Home updates.	Request becomes Cancelled; open offers close; Home updates.	✓	
2	CANCEL-002: Verify reason validation.	Request: REQ-4001; Reason: blank	Confirm is blocked or validation appears.	Confirm is blocked or validation appears.	✓	
3	CANCEL-003: Verify accepted/en route policy.	Request: REQ-4002; Status: Accepted/En route; Reason:	Backend confirms cancellation or returns clear policy error.	Backend confirms cancellation or returns clear policy error.	✓	

		Delay or safety concern				
4	CANCEL-004: Verify cancelled trip appears in Activity.	Cancelled request: REQ-4001	Cancelled move appears with status and details.	Cancelled move appears with status and details.	✓	

Test Scenario Name: Payment

Description: This scenario verifies cash payment confirmation, payment timing restrictions, client payment visibility, and dispute recording.

Table 4.5-8 Test case for Payment

#	Test case	Input Data	Expected Result	Actual Result	P	F
1	PAY-001: Verify owner confirms payment after completion.	Completed request: REQ-5001; Amount: ETB 2,800; Payment method: Cash	Payment becomes confirmed and disappears from pending confirmation list.	Payment becomes confirmed and disappears from pending confirmation list.	✓	
2	PAY-002: Verify blocked early payment.	Active request: REQ-5002; Status: In transit; Payment method: Cash	Action is unavailable or backend rejects.	Action is unavailable or backend rejects.	✓	
3	PAY-003: Verify client sees pending/disputed state.	Completed request: REQ-5001; Payment record: Pending confirmation	Payment state is visible and human-readable.	Payment state is visible and human-readable.	✓	
4	PAY-004: Verify dispute eligibility.	Request: REQ-5001; Dispute reason:	Dispute is recorded and notification/report state updates.	Dispute is recorded and notification/report state updates.	✓	

		Amount disagreement				
--	--	---------------------	--	--	--	--

Test Scenario Name: Rating and Report

Description: This scenario verifies completed-trip rating, duplicate rating control, issue reporting, and evidence handling.

Table 4.5-9 Test case for Rating and Report

#	Test case	Input Data	Expected Result	Actual Result	P	F
1	TRUST-001: Verify rating submission.	Request: REQ-6001; Rating: 5 stars; Note: Driver was punctual	Review saved and owner rating data updates after refetch.	Review saved and owner rating data updates after refetch.	✓	
2	TRUST-002: Verify duplicate rating blocked if backend supports it.	Request: REQ-6001; Existing rating already submitted	Backend blocks duplicate or returns idempotent existing result.	Backend blocks duplicate or returns idempotent existing result.	✓	
3	TRUST-003: Verify report without file.	Category: Damage; Description: Item was scratched; Evidence: none	Report code appears; report is saved.	Report code appears; report is saved.	✓	
4	TRUST-004: Verify optional evidence attachment.	Category: Damage; Description: Box damaged; Evidence: damage_photo_test.jpg	Report saved; evidence attached or retry message shown.	Report saved; evidence attached or retry message shown.	✓	
5	TRUST-005: Verify validation.	Category: Delay; Description: blank	Clear validation error; no report created.	Clear validation error; no report created.	✓	

Test Scenario Name: Notifications

Description: This scenario verifies notification listing, read status, detail navigation, and user notification preferences.

Table 4.5-10 Test case for Notifications

#	Test case	Input Data	Expected Result	Actual Result	P	F
1	NOTIF-001: Verify unread count/list.	Notifications: offer, status update, message; Unread count: 3	Unread count and rows display clearly.	Unread count and rows display clearly.	✓	
2	NOTIF-002: Verify read action.	Notification ID: N-7001; Status: Unread	Notification becomes Read; unread count decreases.	Notification becomes Read; unread count decreases.	✓	
3	NOTIF-003: Verify notification navigation.	Notification ID: N-7002; Request ID: REQ-3001; Type: offer/detail	Owner goes to Offers; client goes to Home/details area.	Owner goes to Offers; client goes to Home/details area.	✓	
4	NOTIF-004: Verify preferences save.	Preferences: Push on; SMS off; Email on	Preferences save and success message appears.	Preferences save and success message appears.	✓	

Test Scenario Name: UI/UX Regression

Description: This scenario verifies common mobile usability, visual consistency, empty states, loading states, error states, and regression behavior.

Table 4.5-11 Test case for UI/UX Regression

#	Test case	Input Data	Expected Result	Actual Result	P	F
1	UI-001: Verify tab navigation and icons.	Client and owner sessions; Tabs: Home, Request, Activity, Notifications,	Tabs use professional icons, labels fit, and navigation works.	Tabs use professional icons, labels fit, and navigation works.	✓	

KULLI: P2P Logistics Mobile Application On Demand Trucking Service

		Vehicles, Offers, Earnings				
2	UI-002: Verify route/debug labels removed.	Screens: Home, Request, Activity, Notifications, Vehicles, Offers, Earnings	No `/client/...`, `/owner/...`, workflow, phase, or debug labels in normal screens.	No `/client/...`, `/owner/...`, workflow, phase, or debug labels in normal screens.	✓	
3	UI-003: Verify loading copy and layout.	Slow network profile; Screen: Notifications list	Loading state is clear and not oversized or confusing.	Loading state is clear and not oversized or confusing.	✓	
4	UI-004: Verify empty states.	Fresh demo account with no requests, offers, vehicles, or notifications	Empty states explain next action and avoid blank space.	Empty states explain next action and avoid blank space.	✓	
5	UI-005: Verify recoverable errors.	API stopped; Invalid form submission on Request screen	Error is user-visible, concise, and recoverable.	Error is user-visible, concise, and recoverable.	✓	
6	UI-006: Verify mobile layout.	Viewport: 360 x 640 px mobile emulator	Text does not overlap; buttons remain tappable; bottom tabs do not cover critical controls.	Text does not overlap; buttons remain tappable; bottom tabs do not cover critical controls.	✓	
7	UI-007: Verify map-first request/tracking .	Route: Atlas to Mexico; Active tracking request available	Map preview is visible; zoom/full-screen controls work where available.	Map preview is visible; zoom/full-screen controls work where available.	✓	
8	UI-008: Verify guided flow.	Route: Bole to CMC; Truck: Medium Truck; Load: 600 kg	Flow is understandable and preserves input after recoverable errors.	Flow is understandable and preserves input after recoverable errors.	✓	

REFERENCES

1. Addis Ababa University, Department of Computer Science, Undergraduate Final Project Guideline.
2. <https://reactnative.dev/docs/getting-started>
3. <https://www.geeksforgeeks.org/unified-modeling-language-uml-sequence-diagrams/>
4. <https://online.visual-paradigm.com/diagrams/tutorials/deployment-diagram-tutorial/>
5. <https://www.trade.gov/country-commercial-guides/ethiopia-transport-and-logistics>
6. <https://www.forbes.com/sites/forbestechcouncil/2021/08/17/the-uberization-of-logistics/>
7. <https://doi.org/10.1016/j.trpro.2020.08.001>
8. <https://www.wrike.com/agile-guide/agile-development-life-cycle/>
9. <https://swimm.io/learn/system-design/system-design-complete-guide-with-patterns-examples-and-techniques>
10. [https://en.wikipedia.org/wiki/Last_mile_\(transportation\)](https://en.wikipedia.org/wiki/Last_mile_(transportation))